L.J SCHOOL OF COMPUTER APPLICATION



LJ University
University with a Difference

LinkPro Networking Platform

Developed By:

Anand Yadav: 22002501210066

Smit Dutt: 22002501210008

Hardik Nakum: 22002501210035

Under the Guidance of:

Prof. Rita Gokani

L.J SCHOOL OF COMPUTER APPLICATION

LJ Campus, LJ University Road, Off S.G. Road, Ahmedabad-382210



CERTIFICATE

Enrolment No: 22002501210066

This is to certify that **Mr.Anand Yadav** of Integrated Master of Computer Applications (IMCA), Semester 9, **Roll No B66**, has satisfactorily completed his project titled **LinkPro** under the supervision of **Prof.Rita Gokani**

Internal Guide Name	**Head of Department**
Prof.Rita Gokani	**DR.Monica Gahalawat**

SIGNATURE

Date of Submission:

DIRECTOR

Prof.Alok Manke

L.J SCHOOL OF COMPUTER APPLICATION

L.J SCHOOL OF COMPUTER APPLICATION

LJ Campus, LJ University Road, Off S.G. Road, Ahmedabad-382210



CERTIFICATE

Enrolment No: 22002501210008

This is to certify that **Mr.Smit Dutt** of Integrated Master of Computer Applications (IMCA), Semester 9, **Roll No B09**, has satisfactorily completed his project titled **LinkPro** under the supervision of **Prof.Rita Gokani**

Internal Guide Name	Head of Department
Prof.Rita Gokani	**DR.Monica Gahalawat**

SIGNATURE

Date of Submission:

DIRECTOR

Prof.Alok Manke

L.J SCHOOL OF COMPUTER APPLICATION

L.J SCHOOL OF COMPUTER APPLICATION

LJ Campus, LJ University Road, Off S.G. Road, Ahmedabad-382210



CERTIFICATE

Enrolment No: 22002501210035

This is to certify that **Mr.Hardik Nakum** of Integrated Master of Computer Applications (IMCA), Semester 9, **Roll No B32**, has satisfactorily completed his project titled **LinkPro** under the supervision of **Prof.Rita Gokani**

Internal Guide Name	**Head of Department**
Prof.Rita Gokani	**DR.Monica Gahalawat**

SIGNATURE

Date of Submission:

DIRECTOR

Prof.Alok Manke

L.J SCHOOL OF COMPUTER APPLICATION

Table of Contents

Sr. No.	Table of Contents	Page No.
1.	**INTRODUCTION**	**6**
	1.1 Project Profile	6
	1.2 Existing System	7
	1.3 Need for the New System	7
	1.4 Problem Definition	7
	1.5 Objective of the New System	7
	1.6 Scope of the Project & Core Components	8
	1.7 Feasibility Study (Technical / Economical / Operational)	9
	1.8 Advantages and Limitations	9
	1.9 Assumptions and Constraints	10
	1.10 Stakeholder Analysis & RACI Matrix	10
2.	**REQUIREMENT DETERMINATION & ANALYSIS**	**11**
	2.1 Functional Requirements	11
	2.2 Non-Functional Requirements	11
	2.3 Targeted Users	11
	2.4 Software Requirement	12
3.	**PROJECT PLANNING & MANAGEMENT**	**13**
	3.1 Project Charter	13
	3.2 Project Lifecycle Model	19
	3.3 Work Breakdown Structure	20
	3.4 Effort Estimation	21
	3.5 Gantt Chart / Sprint Schedule	21
	3.6 Sprint Backlog	23
	3.7 User Stories	25
	3.8 Release Plan	26
	3.9 Risk Register	28
	3.10 Change Request Log	33
	3.11 Status Reports	34
4.	**SYSTEM DESIGN**	**36**
	4.1 System Architecture	36
	4.2 Module Description	38
	4.3 Use Case Diagram	39
	4.4 Class Diagram	40
	4.5 Sequence / Interaction Diagram	41
	4.6 Activity Diagram	43
	4.7 Data Dictionary	44
5.	**DEVELOPMENT**	**46**
	5.1 Coding Standards	46
	5.2 SOLID Principles & Refactoring	47
6.	**DESIGN PATTERN IMPLEMENTATION & DOCUMENTATION**	**53**
	6.1 Pattern Selection & Justification	53
	6.2 Creational Pattern(s) Implementation	54
	6.3 Structural Pattern(s) Implementation	55
	6.4 Behavioral Pattern(s) Implementation	56
	6.5 UML Mapping (Pattern-to-Class Diagram Traceability)	57
	6.6 Documentation & Trade-off Analysis	60
	6.7 Testing	61
	6.8 Project Tracking & Monitoring	63
	6.9 Screenshots	75
	6.10 Proposed Enhancement	81
	6.11 Project Closure Report	81
	6.12 Conclusion	85
	6.13 Bibliography	85

1. Introduction

A full-fledged **LinkedIn-like platform** has been created, allowing professionals to create comprehensive profiles featuring work experience, skills, education, certifications, and achievements, along with personalized profile pictures and background images. Professionals **can find and connect** with other professionals in their industry using personalized suggestions and common connections.

The live news feed displays **real-time** posts, articles, job updates, and company announcements. Professionals can like, comment, share posts, send connection requests, endorse skills, write recommendations, and apply for job postings. **In-app messaging** enables private messaging with read receipts and group messaging for teams.

Powerful search helps find people, jobs, and companies using intelligent filters. Activity feeds track profile views, connection requests, and post engagement. Job seekers can view company profiles that display employee information, salary information, and application status. The platform is **optimized** to handle hundreds of users seamlessly, with instant notifications and a seamless mobile experience.

1.1 Project Profile:-

Project Title	LinkPro Networking Platform
Type of Application	Web Application
Project Overview	LinkPro is a comprehensive professional networking system designed to enhance career opportunities, streamline job search, and facilitate meaningful professional connections. By integrating profile management, real-time newsfeeds, job applications, and advanced networking features, LinkPro overcomes limitations of fragmented job boards and traditional networking methods.
Team Size	3 People
Front End	React.js, Tailwind CSS, JavaScript
Back End	Node.js, Express.js
Database	MongoDB
Tools Used	Microsoft Visual Studio

1.2 Existing System:-

The Existing platforms offer many features, but they are **complex, handle a very large number of users, and may face performance issues during peak usage**. Their search results can be **too broad**, and many advanced features are limited to paid users. Our proposed project is required to provide a **simple, user-friendly, and focused networking platform** with **easy search, better performance, real-time communication, and useful features for users without unnecessary complexity**.

1.3 Need for the New System:-

The new **LinkPro** system is needed to overcome the limitations of existing professional networking platforms. Many users, especially **students and job seekers**, face problems such as **scattered job platforms, limited networking, high subscription costs, and difficult job searches**. Our system aims to provide a **simple, single platform** for creating profiles, finding jobs, connecting with professionals, and communicating in real time. It will be **easy to use, affordable, and focused on the needs of Indian users**, helping them find suitable career opportunities more efficiently.

1.4 Problem Definition:-

The proposed LinkPro system's scope includes an extensive online networking platform that aims to **improve career** efficiency and professional networking access. The key issues that the system addresses include:

- **Time-consuming job searches**: Traditional job applications take a lot of time. LinkPro provides **easy profile creation, quick job applications, and recruiter connections**.
- **Difficult networking:** Finding the right professionals can be difficult. LinkPro brings **professionals, companies, and alumni onto one platform** for easy connections.
- **Geographical limitations:** Users from **tier-2/3 cities and rural areas** may have limited access to opportunities. LinkPro provides **online access to jobs and professionals from anywhere**.
- **High costs:** Existing platforms may require **premium subscriptions or other fees**. LinkPro provides **basic networking and job-search features at low or no cos**t, making it useful for students and early-career professionals.

1.5 Objective of the New System:-

The main objectives of LinkPro are to make **job searching, professional networking, and career growth easier and faster.**

- **Easy job applications:** Users can create profiles and **apply for jobs online**, reducing the time needed for hiring.
- **Professional communication**: Users can **message, collaborate, get advice, and connect with mentors** from anywhere.
- **Smart search:** Users can find **relevant jobs, professionals, and companies** using filters such as **skills, location, experience, and industry**.

1.6 Scope of the Project & Core Components:-

1.6.1 Scope of the Project:-

The scope of **LinkPro** is to develop a **professional networking platform** that brings career networking and job-related activities together in one place. The system allows users to create and manage professional profiles, connect with other professionals, share and interact with posts, communicate through real-time messaging, and receive notifications. It also supports searching for professionals and job opportunities, applying for jobs, and managing applications. An **Admin Panel** is included to manage users and platform content, while the system is designed to be **responsive, secure, and scalable** for future enhancements.

1.6.2 Core Components:-

- **Profile Management Module:-**

Users can create and manage professional profiles with experience, skills, education, etc. This module helps improve personal branding and makes it easy for recruiters to find users.

- **Networking Module:-**

The module helps users connect, send messages, and find common connections. It helps users build professional networks and get referrals easily.

- **Job Management Module:-**

The module allows users to search for jobs, apply to jobs with a single click, and track job applications. It helps reduce the recruitment timeline from weeks to days.

- **Newsfeed Module:-**

The module enables users to post, like, comment, and share content in real-time. It keeps professionals updated about the latest trends and opportunities in their industry.

- **Real-Time Chat Module:-**

This module allows connected users to communicate through real-time messaging. Users can send and receive messages and maintain their conversation history.

- **Notification Module:-**

This module provides real-time notifications for important activities such as connection requests, messages, likes, comments, and other interactions.

- **Admin Management Module:-**

This module allows administrators to manage users, control permissions, and manage platform content, helping maintain the security and proper functioning of LinkPro.

1.7 Feasibility Study (Technical /Economic /Operational):-

The feasibility study evaluates whether the proposed **LinkPro Networking Platform** can be developed, used, and maintained successfully within the project's available technical, economic, and operational conditions.

Feasibility Area	Assessment	Project Evidence	Conclusion
Technical Feasibility	The project is technically feasible using a MERN-based web application architecture.	React.js, Tailwind CSS, JavaScript, Node.js, Express.js, MongoDB, JWT authentication, Socket.io, Postman, ESLint and Prettier are identified in the existing documentation.	Technically Feasible
Economic Feasibility	The project can be developed using commonly available development tools and technologies.	The documented stack and development tools are suitable for an academic project, and the project is designed as a web-based platform.	Economically Feasible
Operational Feasibility	The system is intended to provide a user-friendly platform for profiles, networking, posts, messaging, notifications and job-related activities.	Targeted users include job seekers, recruiters/HR professionals, freelancers/consultants and tier-2/3 city professionals.	Operationally Feasible

1.8 Advantages and Limitations :-

- **Advantages :-**
  - Benefits tier-2/3 city professionals and fresher’s with access to top companies and recruiters previously out of reach.
  - Enables instant profile creation, job applications, and networking—reducing hiring cycles from months to days.
  - Real-time newsfeed and messaging keep professionals updated on opportunitiesand industry trends instantly.
  - Advanced search connects users with precise job matches, mutual connections, and verified professionals.
- **Disadvantages :-**
  - Requires stable internet connectivity.
  - Cannot fully replace in-person networking events, interviews, or physical portfolio presentations.
  - Needs many users to be really useful.

1.9 Assumptions and Constraints:-

- **Assumptions :-**

Experts are increasingly using online networking to improve efficiency and extend their reach. Users have basic digital skills and reliable 4G+ internet connectivity to enable real-time functionality. User profiles must be kept up to date to reflect genuine experience and skills. Employers are actively investigating endorsements and user behavior. Job hunters value shared connections rather than unsolicited communication. Privacy compliance helps to build trust in sharing professional information.

- **Constraints :-**

The platform is constrained by regional hiring laws that influence job posting compliance. Device compatibility problems may arise, affecting mobile responsiveness on older smartphones or slower internet connectivity. The presence of intense competition from existing platforms requires differentiation based on better user experience, greater accuracy in AI-driven matching, and the provision of free premium services.

1.10 Stakeholder Analysis & RACI Matrix:-

1.10.1 Stakeholder Analysis:-

Stakeholder analysis identifies the people and user groups that influence, guide, develop, review, or use the LinkPro platform. The project documentation identifies the project team, internal guide, project manager, administrators, and targeted users as relevant project roles.

Stakeholder	Role / Interest	Influence	Engagement
Project Team	Development, documentation, testing and project execution	High	High
Internal Guide – Prof. Rita Gokani	Academic guidance and project review	High	High
Administrator	User/content management and platform administration	High	High
Job Seekers / Professionals	Use profiles, networking, posts, messaging and job-related functions	Medium	High
LJ University	Academic stakeholder	Medium	Medium

1.10.2 RACI Matrix:-

R = Responsible | A = Accountable | C = Consulted | I = Informed

Activity	Project Manager	Frontend Developer	Backend Developer	Database Developer	User
Requirement determination	[R]	[C]	[C]	[C]	[A]
System design	[A]	[R]	[R]	[R]	[A]
Development & integration	[A]	[R]	[R]	[R]	-
Testing & quality review	[A]	[R]	[R]	[R]	[C]
Project review & academic approval	[R]	[I]	[I]	[I]	-
User/content administration	[A]	[C]	[R]	[C]	[C]
Documentation	[R]	[C]	[C]	[C]	-
Final Project Submission	[R/A]	[I]	[I]	[I]	-

2. Requirement Determination and Analysis.

2.1 Functional Requirements:-

The **functional requirements** describe the major services and operations that LinkPro is expected to provide to its users and administrators.

1. User Registration & Login.
2. Profile Management.
3. Resume Management
4. Networking & Connection Management.
5. Post Creation & Newsfeed
6. Like & Comment.
7. Real-Time Chat & Messaging.
8. Search & Filter.
9. Admin Management.
10. Logout & Account Security.

2.2 Non- Functional Requirements:-

The **non-functional requirements** define the expected quality attributes and operational characteristics of the LinkPro system.

1. Performance – Ensures the system responds quickly and efficiently.
2. Security – Protects user data and prevents unauthorized access.
3. Scalability – Allows the system to handle increasing users and data.
4. Usability – Makes the system easy and convenient to use.
5. Reliability – Ensures the system works correctly and consistently.
6. Maintainability – Makes the system easy to update, fix, and maintain.
7. Availability – Ensures the system is accessible when required.
8. Compatibility – Ensures the system works across supported devices and browsers.
9. Privacy – Protects users' personal and professional information.
10. Responsiveness – Ensures the interface adapts properly to different screen sizes.

2.3 Targeted Users:-

The targeted users of **LinkPro** primarily include:-

- Job Seekers (Freshers, Experienced Professionals, Career Switchers).
- Recruiters and HR Professionals.
- Freelancers and Consultants (project-based opportunities).
- Tier-2/3 City Professionals (access to metro job markets).

2.4 Software Requirement Specification (SRS):-

Component	Requirement
Operating System	Windows 10/11, Linux, or macOS
Frontend	React.js, Tailwind CSS, JavaScript
Backend	Node.js, Express.js
Database	MongoDB
Development Tool	Visual Studio 2022
Version Control	Git & GitHub
Web Browser	Google Chrome, Microsoft Edge, Mozilla Firefox, etc.
API Testing	Postman
Real-Time Communication	Socket.io
Authentication	JWT & bcrypt

3. Project Planning & Management.

3.1 Project Charter:-

Project Name or Identification:-

Item	Details
Project Name / Identification	LinkPro – Professional Networking Platform
Project Type	Web-Based Professional Networking Platform
Development Methodology	Agile
Internal Guide	Prof. Rita Gokani
Project Team	Anand Yadav, Smit Dutt, Hardik Nakum
Institution	LJ University

3.1.1 Project Stakeholders:-

1. Anand Yadav

Role/Responsibility: Backend Developer, Database Integration & Team Manager

Phone Number: +91 90815 05659

E-mail Address: unknown98256@gmail.com

2. Smit Dutt

Role/Responsibility: Frontend Developer

Phone Number: +91 93139 64403

E-mail Address: smitdutt727@gmail.com

3. Hardik Nakum

Role/Responsibility: Documentation & Quality Assurance

Phone Number: +91 96019 77655

E-mail Address: hardikn161@gmail.com

3.1.2 Project Description:-

- **Background**

  LinkPro is a **professional networking platform** designed to provide users with a centralized platform for creating professional profiles, building connections, communicating, sharing content, and exploring career opportunities.

- **Description of the Challenge or Opportunity**

  Traditional professional networking and job-search activities can be **time-consuming** and **distributed** across multiple platforms. LinkPro provides an opportunity to bring professional networking, communication, content sharing, and job-related activities together in one centralized platform.

- **Overview of the Desired Impact:-**

  LinkPro aims to **improve professional networking and career opportunities** by making it easier for users to connect with professionals, discover suitable jobs, communicate in real time, and manage their professional identity. It is intended to make career development more **accessible**, **efficient**, and **organized**.

3.1.3 Measurable Organizational Value (MOV):-

The MOV defines the measurable value expected from the project and focuses on improved networking, career access, communication, professional identity, user engagement, and centralized administration.

MOV Area	Measurement / Indicator	Expected Value
Professional Networking	Number of professional connections / networking interactions	Improved professional networking
Career Opportunities	Job searches and applications supported by the platform	Better access to career opportunities
Communication	Messages and conversations supported	Faster professional communication
Professional Identity	Profiles containing skills, education and experience	Stronger professional presence
User Engagement	Posts, likes, comments and interactions	Higher user engagement
Administration	User and content management activities	Centralized platform management

3.1.4 Project Scope:-

- **What Will Be Included in the Scope**
  - User registration, login and secure authentication
  - Professional profile creation and management
  - Professional networking and connection requests
  - Posts, newsfeed, likes and comments
  - Real-time messaging and communication
  - Job search, job posting, applications and application tracking
  - Search and filtering
  - Notifications
  - Admin user and content management
  - Responsive web interface

- **What Will Be Considered Outside the Scope**
  - Native Android/iOS mobile applications
  - Advanced AI-based recruitment and recommendation features
  - Advanced third-party recruitment integrations
  - Features not defined in the approved project requirements

3.1.5 Project Schedule Summary:-

Project Start Date: 1 August 2026.

Project End Date: 31 October 2026.

- **Timeline of Project Phases and Milestones**

Phase	Duration	Milestone
Project Initiation & Requirement Analysis	August 2026	Project topic finalized, requirements gathered, feasibility study completed, and Project Charter approved.
System Design	August 2026	UI/UX design, database schema, use case diagram, class diagram, activity diagram, and system architecture completed.
Development	September 2026	Frontend, backend, database integration, authentication, admin panel, language learning modules, videos, MCQs, and progress tracking implemented.
Testing & Quality Assurance	October 2026	Functional testing, integration testing, bug fixing, and user acceptance testing completed.
Documentation & Final Submission	October 2026	Final documentation, project deployment, presentation, demonstration, and project submission completed.

- **Project Reviews and Review Dates**

Review	Tentative Date
Project Proposal Review	August 2026
Requirement & Design Review	August 2026
Development Progress Review – I	September 2026
Development Progress Review – II	September 2026
Testing & Quality Review	October 2026
Final Project Review & Submission	October 2026

3.1.6 Project Budget Summary:-

This project is developed as an **academic project** for educational and learning purposes.

3.1.7 Quality Issues:-

Specific quality requirements for LinkPro include:

• Functional Correctness – Ensures all features work according to the defined requirements.

• Security – Protects user information and authentication data from unauthorized access.

• Performance – Ensures fast and efficient system response.

• Usability – Provides a simple and user-friendly interface.

• Reliability – Ensures consistent and error-free system operation.

• Maintainability – Makes the system easy to update, modify, and maintain.

• Compatibility – Ensures proper operation across supported browsers and devices.

• Responsive Design – Ensures the interface adapts properly to different screen sizes.

3.1.8 Resources Required:-

Resource Category	Resources	Purpose
People	Project Team, Project Manager, Internal Guide, Administrator	Planning, development, review and administration
Technology	React.js, Tailwind CSS, JavaScript, Node.js, Express.js, MongoDB, Socket.io	System development
Security	JWT, bcrypt, environment variables	Authentication and protection
Software Tools	Visual Studio Code / Visual Studio, Git, GitHub, Postman, ESLint, Prettier	Development, version control, testing and quality
Facilities	Computers, internet connectivity, testing devices	Development and testing

- **Resources to be Provided:-**

Resource	Resource Provider	Date to be Provided
Project Guidance	Prof. Rita Gokani	July 2025
Development Laptop/Desktop	Team Members	July 2025
Internet Connection	Team Members	July 2025
Development Tools (VS Code, Git, Postman)	Team Members	July 2025
Database	MongoDB Atlas (Free Tier)	August 2025
College Computer Laboratory	L.J. Institute of Computer Applications	Throughout the Project

3.1.9 Assumptions and Risks:-

- **Assumptions Used to Develop Estimates:-**

1. Required development tools and technologies will be available.
2. Project team members will be available during planned activities.
3. Internet connectivity will be available for development, testing and real-time features.
4. The approved requirements will remain reasonably stable during the project.

- **Key Risks, Probability and Impact:-**

Risk	Probability	Impact	Risk Level	Response
Internet connectivity issues	Medium	High	High	Optimize and test under varying network conditions.
Browser / device compatibility	Medium	Medium	Medium	Test supported browsers and devices.
Scalability with increasing users	Medium	High	High	Use modular architecture and optimize APIs.
Security / unauthorized access	Medium	High	High	Use secure authentication, authorization and password hashing.
Integration defects	Medium	High	High	Perform continuous integration and testing.
Privacy / legal requirements	Medium	High	High	Review applicable requirements before deployment.

- **Constraints:-**

• Limited academic project timeline
• Limited development resources
• Dependence on internet connectivity
• Availability of team members and facilities

- **Dependencies on Other Projects or Areas:-**

• Availability of MongoDB database services.
• Availability of development tools (Visual Studio Code, Git, Postman).
• Continuous guidance and feedback from the project guide.
• Availability of internet connectivity for development and testing.

- **Assessment of Project Impact on the Organization:-**

• Improves Professional Networking – Provides an accessible digital platform for users to build professional connections and expand their network.
• Enhances Career Opportunities – Helps users discover suitable jobs, apply for opportunities, and manage their professional profiles.
• Promotes Digital Professional Development – Provides practical experience in modern web technologies and demonstrates the application of the MERN stack in a real-world project.
• Centralizes Professional Activities – Brings profiles, networking, communication, posts, notifications, and job-related activities together on a single platform.

- **Outstanding Issues:-**

• Content and User Data Management – Continuous management and validation of professional profiles, job information, and platform content.
• Advanced Recommendation Features – Future implementation of AI-based job and professional recommendations can improve user experience.
• Platform Expansion – Future versions can include additional features and services to support a wider range of professional networking needs.
• Deployment and Long-Term Maintenance – Final deployment, regular updates, security improvements, and long-term maintenance will be required after project completion.

3.1.10 Project Administration:-

- **Communications Plan:-**

Activity	Frequency
Team Meeting	Weekly
Project Guide Review	Bi-weekly
Progress Updates	Weekly
GitHub Repository Updates	Daily

- **Scope Management Plan:-**

• Any new feature request must be approved by the Project Guide.
• Major scope changes will be documented before implementation.
• Features outside the approved scope will be considered for future releases.

- **Quality Management Plan:-**

• Unit Testing
• Integration Testing
• User Acceptance Testing (UAT)
• Code Reviews
• Documentation Reviews

- **Change Management Plan:-**

• Record all change requests.
• Analyze the impact on project scope, schedule, and resources.
• Obtain approval from the Project Guide.
• Update project documentation.
• Implement, test, and verify the approved changes.

- **Human Resources Plan:-**

Role	Primary Responsibility	Involvement
Project Team	Development, testing, documentation and implementation	High
Internal Guide	Academic guidance and review	High

- **Implementation and Project Closure Plan:-**

  • Complete system development.
  • Perform functional, integration, and user acceptance testing.
  • Resolve identified defects.
  • Prepare project documentation.
  • Demonstrate the completed application.
  • Submit the final project report and source code.
  • Obtain final approval from the Project Guide.
  • Officially close the project.

- **3.1.11 Acceptance and Approval:-**

Name	Role	Signature	Date
Prof. Rita Gokani	Project Guide		_____
Anand Yadav	Team Manager		_____
Smit Dutt	Team Member		_____
Hardik Nakum	Team Member		_____

3.2 Project Lifecycle Model (Agile):-

Scrum Model

The **LinkPro** follows the **Scrum Model**, an Agile framework based on iterative and incremental development. The project is divided into multiple **Sprints**, where each Sprint focuses on planning, developing, integrating, testing, and reviewing a specific set of system features.

The Scrum approach allows the development team to **continuously monitor project progress**, **identify and resolve issues, gather user feedback**, and **improve the system** in subsequent Sprints. This approach is suitable for the project because it consists **of multiple modules and functionalities that can be developed and tested incrementally**. The Scrum Model also helps the team **manage changing requirements**, **maintain regular communication among team members**, and **ensure that each Sprint delivers a functional and tested part of the system**.

3.3 Work Breakdown Structure (WBS):-

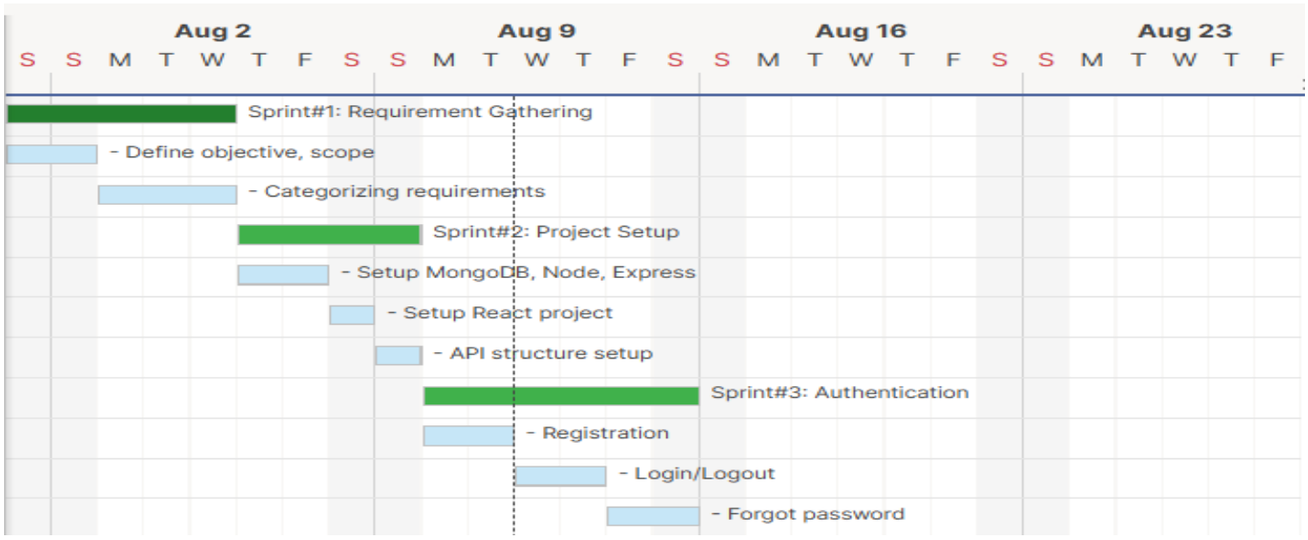
WBS ID	Work Package	Major Activities
1.0	**Requirement Gathering**	1.1 Define project objectives and scope 1.2 Gather functional and non-functional requirements 1.3 Identify target users 1.4 Milestone: Requirements finalized
2.0	**Project Setup**	2.1 Set up React, Node.js and Express.js 2.2 Configure MongoDB database 2.3 Set up GitHub repository and project structure 2.4 Milestone: Project environment configured
3.0	**Authentication**	3.1 User registration and login 3.2 OTP/authentication and password recovery 3.3 Logout and access management 3.4 Milestone: Authentication completed
4.0	**Profile Management**	4.1 Create and display user profile 4.2 Edit profile information and profile image 4.3 Milestone: Profile management completed
5.0	**Posts & Feed**	5.1 Create, edit and delete posts 5.2 Develop user feed 5.3 Implement likes and comments 5.4 Milestone: Posts and feed completed
6.0	**Connections**	6.1 Send connection requests 6.2 Accept/reject connection requests 6.3 Manage connections 6.4 Milestone: Connections completed
7.0	**Chat System**	7.1 Develop real-time chat functionality 7.2 Implement chat UI and message storage 7.3 Milestone: Chat system completed
8.0	**Jobs Feature**	8.1 Create and display job postings 8.2 Implement job applications 8.3 Milestone: Jobs feature completed
9.0	**Search & Filter**	9.1 Implement user and content search 9.2 Implement filtering functionality 9.3 Milestone: Search and filter completed
10.0	**Notifications**	10.1 Implement notification generation and storage 10.2 Display user notifications 10.3 Milestone: Notifications completed
11.0	**Admin Panel**	11.1 Admin authentication 11.2 User and content management 11.3 Admin dashboard and platform monitoring 11.4 Milestone: Admin panel completed
12.0	**Testing & Integration**	12.1 Functional and integration testing 12.2 API and database testing using Postman 12.3 Bug fixing and final validation 12.4 Milestone: Testing completed
13.0	**UI Improvement**	13.1 Improve interface and navigation 13.2 Implement responsive design 13.3 Milestone: UI improvement completed
14.0	**Deployment**	14.1 Configure production environment 14.2 Deploy frontend, backend and database 14.3 Perform final verification 14.4 Milestone: System successfully deployed

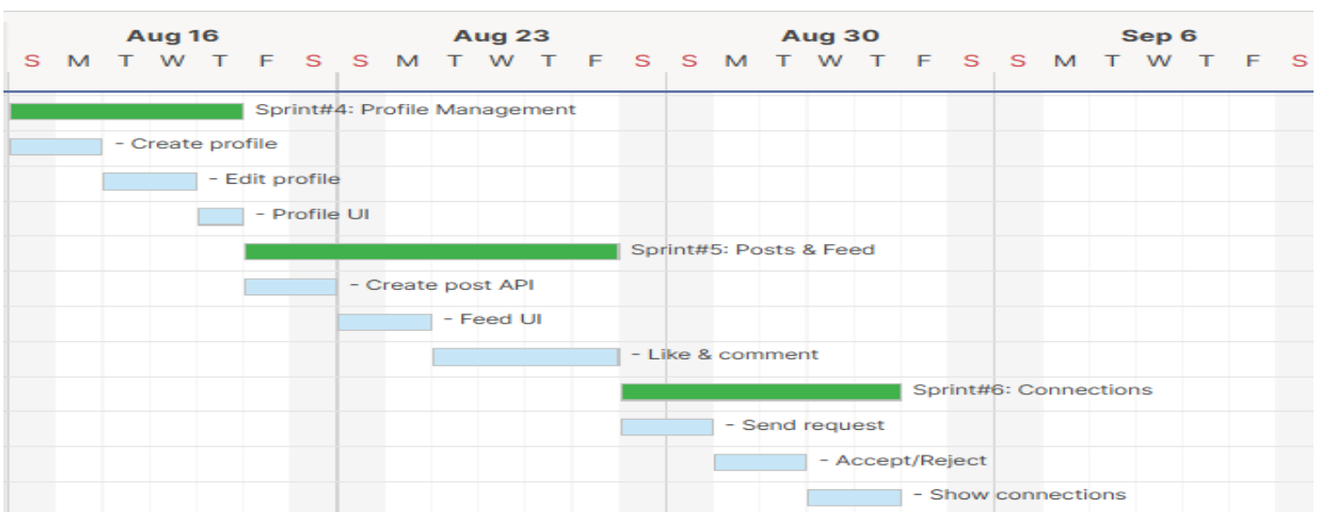
3.4 Effort Estimation:-

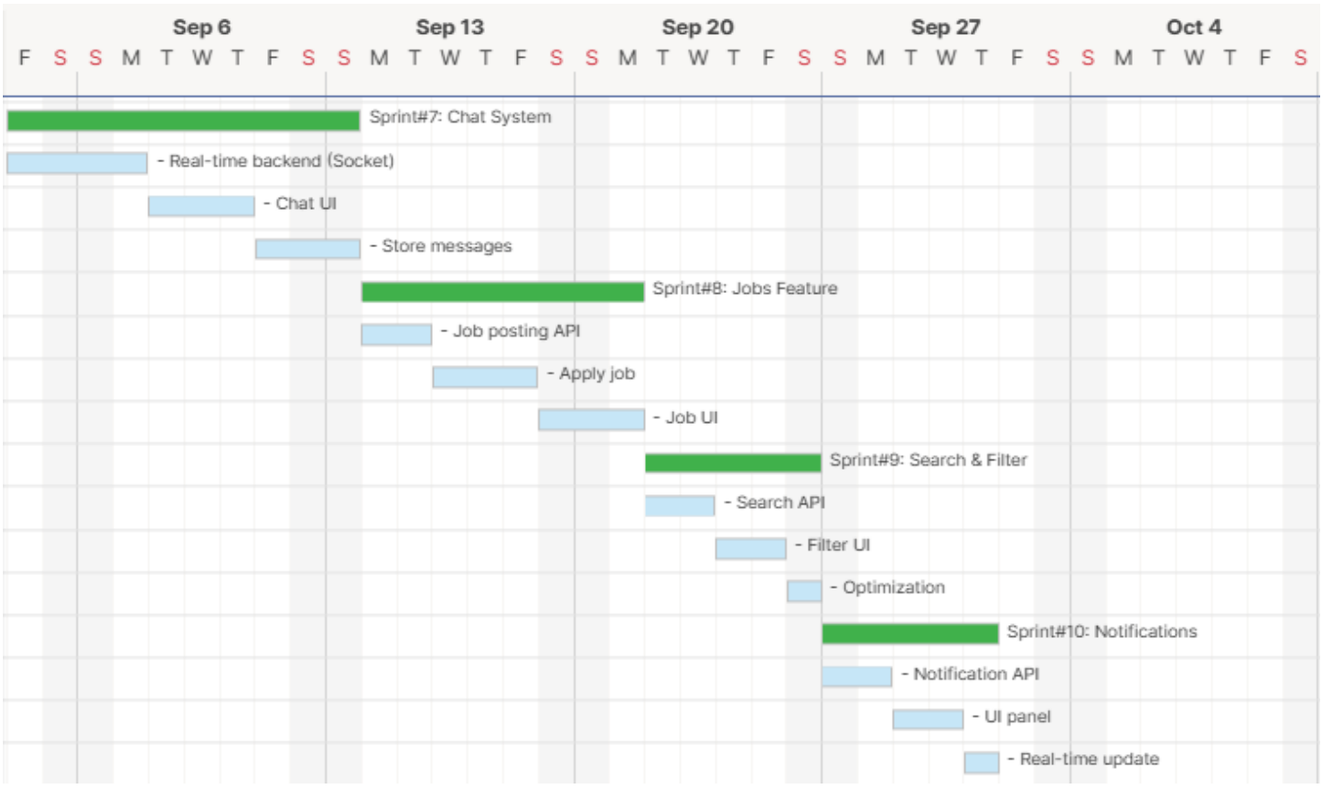
Effort estimation defines the expected duration required to complete each project sprint and activity. The Agile project plan provides durations, start dates, and completion dates for the project activities.

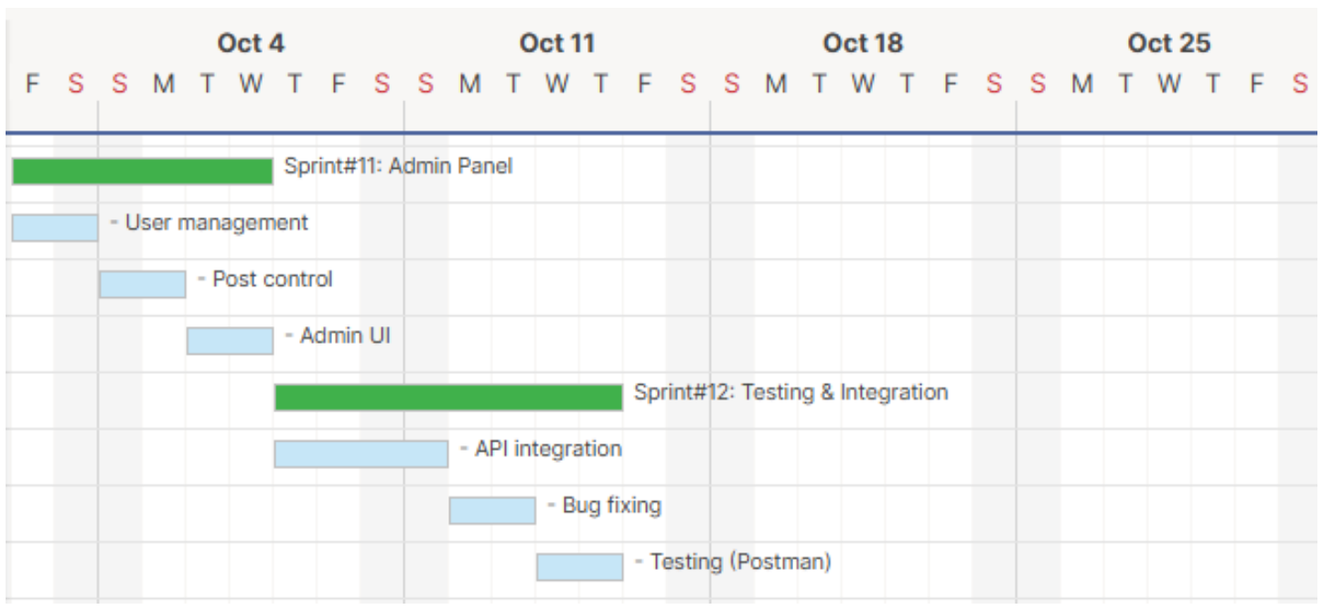
Work Package / Deliverable	Complexity Level	Estimated Effort (Days)	No. of People	Required Roles
Requirements & Setup	Medium	11 days	3	Project Manager, Frontend Developer, Backend Developer
Core User Features	High	31 days	3	Frontend Developer, Backend Developer, Database Developer
Communication & Job Features	High	22 days	2	Frontend Developer, Backend Developer
Search, Notifications & Admin	High	20 days	2	Frontend Developer, Backend Developer
Testing & UI Improvement	Medium	18 days	3	Frontend Developer, Backend Developer, Project Manager
Deployment & Finalization	Medium	13 days	2	Backend Developer, Project Manager
Complete LinkPro Project	**High**	**115 days**	**3**	**Project Team**

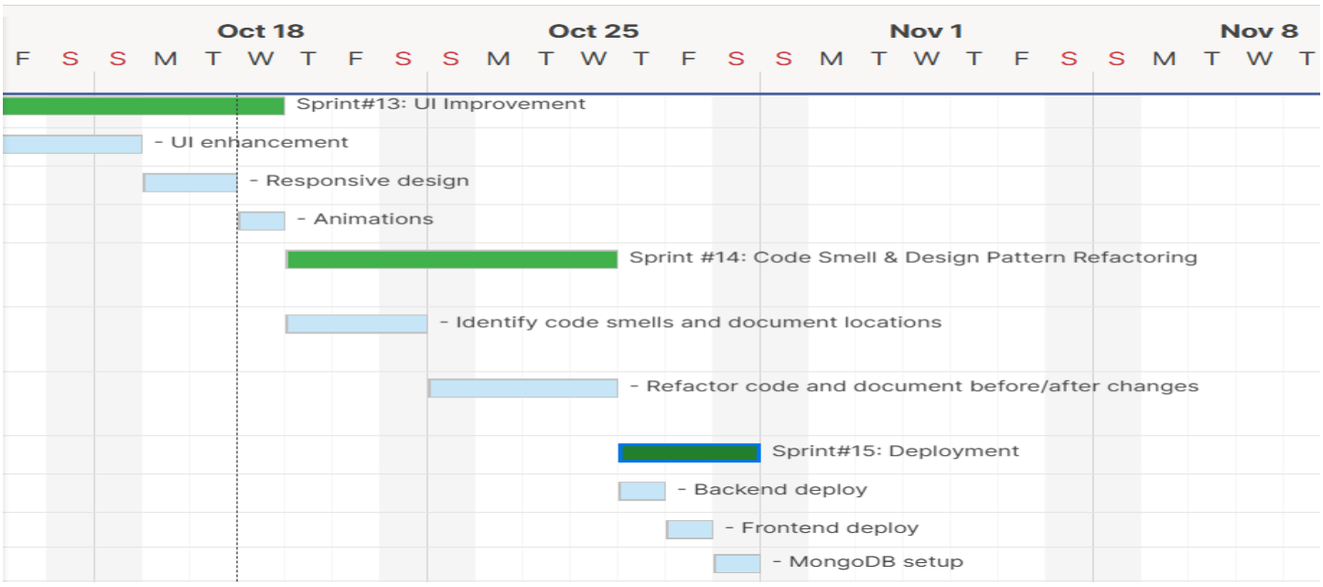
3.5 Gantt Chart / Sprint Schedule:-






Sep 6
Sep 13
Sep 20
Sep 27
Oct 4
F S S M T W T F S S M T W T F S S M T W T F S S M T W T F S S M T W T F S
Sprint#7: Chat System
- Real-time backend (Socket)
- Chat UI
- Store messages
Sprint#8: Jobs Feature
- Job posting API
- Apply job
- Job UI
Sprint#9: Search & Filter
- Search API
- Filter UI
- Optimization
Sprint#10: Notifications
- Notification API
- UI panel
- Real-time update


Oct 4
Oct 11
Oct 18
Oct 25
F S S M T W T F S S M T W T F S S M T W T F S S M T W T F S
Sprint#11: Admin Panel
- User management
- Post control
- Admin UI
Sprint#12: Testing & Integration
- API integration
- Bug fixing
- Testing (Postman)


Oct 18
Oct 25
Nov 1
Nov 8
F S S M T W T F S S M T W T F S S M T W T F S S M T W T
Sprint#13: UI Improvement
- UI enhancement
- Responsive design
- Animations
Sprint #14: Code Smell & Design Pattern Refactoring
- Identify code smells and document locations
- Refactor code and document before/after changes
Sprint#15: Deployment
- Backend deploy
- Frontend deploy
- MongoDB setup

3.6 Sprint Backlog:-

Task Name	Duration	Start	Finish	Status
Sprint#1: Requirement Gathering	**5d**	**01-Aug-2026**	**05-Aug-2026**	**Complete**
- Define objective, scope	2d	01-Aug-2026	02-Aug-2026	Complete
- Categorizing requirements	3d	03-Aug-2026	05-Aug-2026	Complete
Sprint#2: Project Setup	**4d**	**06-Aug-2026**	**09-Aug-2026**	**Complete**
- Setup MongoDB, Node, Express	2d	06-Aug-2026	07-Aug-2026	Complete
- Setup React project	1d	08-Aug-2026	08-Aug-2026	Complete
- API structure setup	1d	09-Aug-2026	09-Aug-2026	Complete
Sprint#3: Authentication	**6d**	**10-Aug-2026**	**15-Aug-2026**	**Complete**
- Registration	2d	10-Aug-2026	11-Aug-2026	Complete
- Login/Logout	2d	12-Aug-2026	13-Aug-2026	Complete
- Forgot password	2d	14-Aug-2026	15-Aug-2026	Complete
Sprint#4: Profile Management	**5d**	**16-Aug-2026**	**20-Aug-2026**	**Complete**
- Create profile	2d	16-Aug-2026	17-Aug-2026	Complete
- Edit profile	2d	18-Aug-2026	19-Aug-2026	Complete
- Profile UI	1d	20-Aug-2026	20-Aug-2026	Complete
Sprint#5: Posts & Feed	**8d**	**21-Aug-2026**	**28-Aug-2026**	**Complete**
- Create post API	2d	21-Aug-2026	22-Aug-2026	Complete
- Feed UI	2d	23-Aug-2026	24-Aug-2026	Complete
- Like & comment	4d	25-Aug-2026	28-Aug-2026	Complete
Sprint#6: Connections	**6d**	**29-Aug-2026**	**03-Sep-2026**	**Complete**
- Send request	2d	29-Aug-2026	30-Aug-2026	Complete
- Accept/Reject	2d	31-Aug-2026	01-Sep-2026	Complete
- Show connections	2d	02-Sep-2026	03-Sep-2026	Complete
Sprint#7: Chat System	**10d**	**04-Sep-2026**	**13-Sep-2026**	**Complete**
- Real-time backend (Socket)	4d	04-Sep-2026	07-Sep-2026	Complete
- Chat UI	3d	08-Sep-2026	10-Sep-2026	Complete
- Store messages	3d	11-Sep-2026	13-Sep-2026	Complete
Sprint#8: Jobs Feature	**8d**	**14-Sep-2026**	**21-Sep-2026**	**Complete**
- Job posting API	2d	14-Sep-2026	15-Sep-2026	Complete
- Apply job	3d	16-Sep-2026	18-Sep-2026	Complete
- Job UI	3d	19-Sep-2026	21-Sep-2026	Complete
Sprint#9: Search & Filter	**5d**	**22-Sep-2026**	**26-Sep-2026**	**Complete**
- Search API	2d	22-Sep-2026	23-Sep-2026	Complete
- Filter UI	2d	24-Sep-2026	25-Sep-2026	Complete
- Optimization	1d	26-Sep-2026	26-Sep-2026	Complete
Sprint#10: Notifications	**5d**	**27-Sep-2026**	**01-Oct-2026**	**Complete**
- Notification API	2d	27-Sep-2026	28-Sep-2026	Complete
- UI panel	2d	29-Sep-2026	30-Sep-2026	Complete
- Real-time update	1d	01-Oct-2026	01-Oct-2026	Complete
Sprint#11: Admin Panel	**6d**	**02-Oct-2026**	**07-Oct-2026**	**Complete**
- User management	2d	02-Oct-2026	03-Oct-2026	Complete
- Post control	2d	04-Oct-2026	05-Oct-2026	Complete
- Admin UI	2d	06-Oct-2026	07-Oct-2026	Complete
Sprint#12: Testing & Integration	**8d**	**08-Oct-2026**	**15-Oct-2026**	**Complete**
- API integration	4d	08-Oct-2026	11-Oct-2026	Complete
- Bug fixing	2d	12-Oct-2026	13-Oct-2026	Complete
- Testing (Postman)	2d	14-Oct-2026	15-Oct-2026	Complete

Task Name	Duration	Start	Finish	Status
Sprint#13: UI Improvement	**6d**	**16-Oct-2026**	**21-Oct-2026**	**Complete**
- UI enhancement	3d	16-Oct-2026	18-Oct-2026	Complete
- Responsive design	2d	19-Oct-2026	20-Oct-2026	Complete
- Animations	1d	21-Oct-2026	21-Oct-2026	Complete
Sprint #14: Code Smell & Design Pattern Refactoring	**7d**	**22-10-2026**	**28-10-2026**	**Complete**
- Identify code smells and document locations	3d	22-10-2026	24-10-2026	Complete
- Refactor code and document before/after changes	4d	25-10-2026	28-10-2026	Complete
Sprint#15: Deployment	**3d**	**29-Oct-2026**	**31-Oct-2026**	**Complete**
- Backend deploy	1d	29-Oct-2026	29-Oct-2026	Complete
- Frontend deploy	1d	30-Oct-2026	30-Oct-2026	Complete
- MongoDB setup	½d	31-Oct-2026	31-Oct-2026	Complete
- Final testing	½d	31-Oct-2026	31-Oct-2026	Complete

3.7 User Stories:-

Story-1(Admin)

User Story Id	As a <User>	Want To<Task>	So that<Goal>
1	Admin	Login	I can access the system and manage the platform securely.
2	Admin	View Dashboard	I can see overall system statistics and activities.
3	Admin	Manage User Accounts	I can block, unblock, or delete users within the system.
4	Admin	Control Permissions	I can assign roles and permissions to users.
5	Admin	Logout	I can securely exit the system and protect sensitive information.

Story-2(User)

User Story Id	As a <User>	Want To<Task>	So that<Goal>
1	User	Register	I can create a new account and use the platform.
2	User	Login	I can access my account securely.
3	User	Create Profile	I can add my personal and professional details.
4	User	Edit Profile	I can update my information anytime.
5	User	Add Links/Posts	I can share my content or useful links with others.
6	User	View Other Profiles	I can explore other users and their content.
7	User	Follow/Connect Users	I can build my network on the platform.
8	User	Like/Comment on Posts	I can interact with other users' content.
9	User	Search Users	I can easily find relevant people or posts.
10	User	Send Messages	I can communicate with other users in real-time
11	User	Receive Messages	I can get messages from other users instantly.
12	User	Logout	I can securely exit my account and protect my data.

3.8 Release Plan:-

Task Name	Duration	Start	Finish	Status	Release Date
Sprint#1: Requirement Gathering	**5d**	**01-Aug-2026**	**05-Aug-2026**	**Complete**	**05-Aug-2026**
- Define objective, scope	2d	01-Aug-2026	02-Aug-2026	Complete	02-Aug-2026
- Categorizing requirements	3d	03-Aug-2026	05-Aug-2026	Complete	05-Aug-2026
Sprint#2: Project Setup	**4d**	**06-Aug-2026**	**09-Aug-2026**	**Complete**	**09-Aug-2026**
- Setup MongoDB, Node, Express	2d	06-Aug-2026	07-Aug-2026	Complete	07-Aug-2026
- Setup React project	1d	08-Aug-2026	08-Aug-2026	Complete	08-Aug-2026
- API structure setup	1d	09-Aug-2026	09-Aug-2026	Complete	09-Aug-2026
Sprint#3: Authentication	**6d**	**10-Aug-2026**	**15-Aug-2026**	**Complete**	**15-Aug-2026**
- Registration	2d	10-Aug-2026	11-Aug-2026	Complete	11-Aug-2026
- Login/Logout	2d	12-Aug-2026	13-Aug-2026	Complete	13-Aug-2026
- Forgot password	2d	14-Aug-2026	15-Aug-2026	Complete	15-Aug-2026
Sprint#4: Profile Management	**5d**	**16-Aug-2026**	**20-Aug-2026**	**Complete**	**20-Aug-2026**
- Create profile	2d	16-Aug-2026	17-Aug-2026	Complete	17-Aug-2026
- Edit profile	2d	18-Aug-2026	19-Aug-2026	Complete	19-Aug-2026
- Profile UI	1d	20-Aug-2026	20-Aug-2026	Complete	20-Aug-2026
Sprint#5: Posts & Feed	**8d**	**21-Aug-2026**	**28-Aug-2026**	**Complete**	**28-Aug-2026**
- Create post API	2d	21-Aug-2026	22-Aug-2026	Complete	22-Aug-2026
- Feed UI	2d	23-Aug-2026	24-Aug-2026	Complete	24-Aug-2026
- Like & comment	4d	25-Aug-2026	28-Aug-2026	Complete	28-Aug-2026
Sprint#6: Connections	**6d**	**29-Aug-2026**	**03-Sep-2026**	**Complete**	**03-Sep-2026**
- Send request	2d	29-Aug-2026	30-Aug-2026	Complete	30-Aug-2026
- Accept/Reject	2d	31-Aug-2026	01-Sep-2026	Complete	01-Sep-2026
- Show connections	2d	02-Sep-2026	03-Sep-2026	Complete	03-Sep-2026
Sprint#7: Chat System	**10d**	**04-Sep-2026**	**13-Sep-2026**	**Complete**	**13-Sep-2026**
- Real-time backend (Socket)	4d	04-Sep-2026	07-Sep-2026	Complete	07-Sep-2026
- Chat UI	3d	08-Sep-2026	10-Sep-2026	Complete	10-Sep-2026
- Store messages	3d	11-Sep-2026	13-Sep-2026	Complete	13-Sep-2026
Sprint#8: Jobs Feature	**8d**	**14-Sep-2026**	**21-Sep-2026**	**Complete**	**21-Sep-2026**
- Job posting API	2d	14-Sep-2026	15-Sep-2026	Complete	15-Sep-2026
- Apply job	3d	16-Sep-2026	18-Sep-2026	Complete	18-Sep-2026
- Job UI	3d	19-Sep-2026	21-Sep-2026	Complete	21-Sep-2026
Sprint#9: Search & Filter	**5d**	**22-Sep-2026**	**26-Sep-2026**	**Complete**	**26-Sep-2026**
- Search API	2d	22-Sep-2026	23-Sep-2026	Complete	23-Sep-2026
- Filter UI	2d	24-Sep-2026	25-Sep-2026	Complete	25-Sep-2026
- Optimization	1d	26-Sep-2026	26-Sep-2026	Complete	26-Sep-2026
Sprint#10: Notifications	**5d**	**27-Sep-2026**	**01-Oct-2026**	**Complete**	**01-Oct-2026**
- Notification API	2d	27-Sep-2026	28-Sep-2026	Complete	28-Sep-2026
- UI panel	2d	29-Sep-2026	30-Sep-2026	Complete	30-Sep-2026
- Real-time update	1d	01-Oct-2026	01-Oct-2026	Complete	01-Oct-2026
Sprint#11: Admin Panel	**6d**	**02-Oct-2026**	**07-Oct-2026**	**Complete**	**07-Oct-2026**
- User management	2d	02-Oct-2026	03-Oct-2026	Complete	03-Oct-2026
- Post control	2d	04-Oct-2026	05-Oct-2026	Complete	05-Oct-2026
- Admin UI	2d	06-Oct-2026	07-Oct-2026	Complete	07-Oct-2026

Task Name	Duration	Start	Finish	Status	Release Date
Sprint#12: Testing & Integration	**6d**	**16-Oct-2026**	**21-Oct-2026**	**Complete**	**21-Oct-2026**
- API integration	3d	16-Oct-2026	18-Oct-2026	Complete	18-Oct-2026
- Bug fixing	2d	19-Oct-2026	20-Oct-2026	Complete	20-Oct-2026
- Testing (Postman)	1d	21-Oct-2026	21-Oct-2026	Complete	21-Oct-2026
Sprint#13: UI Improvement	**5d**	**22-Oct-2026**	**26-Oct-2026**	**Complete**	**26-Oct-2026**
- UI enhancement	2d	22-Oct-2026	23-Oct-2026	Complete	23-Oct-2026
- Responsive design	2d	24-Oct-2026	25-Oct-2026	Complete	24-Oct-2026
- Animations	1d	26-Oct-2026	26-Oct-2026	Complete	26-Oct-2026
Sprint #14: Code Smell & Design Pattern Refactoring	**2d**	**27-10-2026**	**28-10-2026**	**Complete**	**28-10-2026**
- Identify code smells and document locations	1d	27-10-2026	27-10-2026	Complete	27-10-2026
- Refactor code and document before/after changes	1d	28-10-2026	28-10-2026	Complete	28-10-2026
Sprint#15: Deployment	**3d**	**29-Oct-2026**	**31-Oct-2026**	**Complete**	**31-Oct-2026**
- Backend deploy	6d	16-Oct-2026	21-Oct-2026	Complete	21-Oct-2026
- Frontend deploy	3d	16-Oct-2026	18-Oct-2026	Complete	18-Oct-2026
- MongoDB setup	2d	19-Oct-2026	20-Oct-2026	Complete	20-Oct-2026
- Final testing	1d	21-Oct-2026	21-Oct-2026	Complete	21-Oct-2026

3.9 Risk Register:-

Risk R-001

Field	Details
Risk ID	R-001
Description	Requirements may change or scope may expand during sprint execution.
Category	Organizational
Root Cause	Evolving stakeholder expectations, agile user feedback loops, and scope creep in social networking features.
Probability / Impact	Medium / High
Risk Score	6
Risk Owner	Project Manager (Anand Yadav)
Response Strategy	Mitigate
Status	Monitoring

Risk R-002

Field	Details
Risk ID	R-002
Description	Network instability and variable internet connectivity impacting real-time communication.
Category	Technical
Root Cause	Intermittent mobile/WiFi disconnections and slow socket handshakes during live messaging.
Probability / Impact	Medium / High
Risk Score	6
Risk Owner	Backend Developer (Anand Yadav)
Response Strategy	Mitigate
Status	Monitoring

Risk R-003

Field	Details
Risk ID	R-003
Description	High concurrent user traffic causing feed query latency and server resource exhaustion.
Category	Technical
Root Cause	Unindexed MongoDB queries, complex multi-user feed aggregations, and missing pagination bounds.
Probability / Impact	Medium / High
Risk Score	6
Risk Owner	Backend Developer & Database Integration (Anand Yadav)
Response Strategy	Mitigate
Status	Open

Risk R-004

Field	Details
Risk ID	R-004
Description	Authentication vulnerabilities, token tampering, or unauthorized profile access.
Category	Technical
Root Cause	Flawed JWT verification middleware, insecure cookie configuration, or improper session expiration.
Probability / Impact	Low / High
Risk Score	3
Risk Owner	Backend Developer (Anand Yadav)
Response Strategy	Mitigate
Status	Monitoring

Risk R-005

Field	Details
Risk ID	R-005
Description	Media upload timeouts, large image payloads, or Cloudinary API rate limits.
Category	Technical
Root Cause	Uncompressed user image uploads, third-party CDN latency, or network bandwidth saturation.
Probability / Impact	Medium / Medium
Risk Score	4
Risk Owner	Backend Developer (Anand Yadav)
Response Strategy	Mitigate
Status	Open

Risk R-006

Field	Details
Risk ID	R-006
Description	Cross-browser rendering bugs and mobile viewport layout inconsistencies.
Category	Technical
Root Cause	CSS Flexbox/Grid discrepancies across browsers (Chrome, Firefox, Safari) and diverse screen resolutions.
Probability / Impact	Medium / Medium
Risk Score	4
Risk Owner	Frontend Developer (Smit Dutt)
Response Strategy	Mitigate
Status	Monitoring

Risk R-007

Field	Details
Risk ID	R-007
Description	Third-party SMTP email delivery failures affecting registration OTPs and password resets.
Category	Technical
Root Cause	Nodemailer SMTP authentication errors, outbound mail port blocks, or mail vendor rate limiting.
Probability / Impact	Medium / High
Risk Score	6
Risk Owner	Backend Developer (Anand Yadav)
Response Strategy	Mitigate
Status	Open

Risk R-008

Field	Details
Risk ID	R-008
Description	User privacy exposure, connection relationship leaks, or unauthorized direct messaging.
Category	Operational / Compliance
Root Cause	Missing route authorization checks, insecure direct object references (IDOR), or permissive API payloads.
Probability / Impact	Low / High
Risk Score	3
Risk Owner	Project Manager (Anand Yadav)
Response Strategy	Mitigate
Status	Monitoring

Risk R-009

Field	Details
Risk ID	R-009
Description	Integration and functional testing exceeding planned sprint timeframes.
Category	Operational
Root Cause	Unexpected regression defects discovered during end-to-end integration across frontend, APIs, and database.
Probability / Impact	Medium / High
Risk Score	6
Risk Owner	QA & Testing (Hardik Nakum)
Response Strategy	Mitigate
Status	Monitoring

Risk R-010

Field	Details
Risk ID	R-010
Description	Production cloud deployment, MongoDB Atlas clustering, and SSL configuration delays.
Category	Operational
Root Cause	Server environment configuration mismatches, missing production environment variables, or build pipeline errors.
Probability / Impact	Low / High
Risk Score	3
Risk Owner	Project Manager (Anand Yadav)
Response Strategy	Mitigate
Status	Open

3.10 Change Request Log

The Change Request Log documents all formal modifications, scope refinements, and architectural additions requested, reviewed, and approved during the LinkPro engineering lifecycle:

CR ID	Description	Submitted By / Date	Impact Assessment	Decision	Status
CR-001	Add GoF Design Patterns across backend subsystems (Singleton, Factory, Facade, Strategy, Builder, State, Observer, Adapter, Decorator).	Project Team August 2026	High impact on backend architecture, code maintainability, and testing procedures.	Approved	Approved
CR-002	Integrate Cloudinary CDN service for real-time post image uploading and user profile avatar processing.	Project Team August 2026	Medium impact on media handling pipeline and client upload latency.	Approved	Approved
CR-003	Implement Socket.IO WebSocket bi-directional transport for instant real-time chat and live unread notification badge updates.	Project Team September 2026	High impact on real-time event decoupling, connection polling, and server memory.	Approved	Approved
CR-004	Add comprehensive Data Dictionary, automated Postman test suites (Test 101–106), and strict RACI stakeholder matrix.	Project Team October 2026	High impact on project auditability, academic compliance, and quality verification.	Approved	Approved

3.11 Status Reports

Status reports document monthly milestone achievements, technical deliverables, team allocations, risks, and next steps across the LinkPro project lifecycle:

3.11.1 Status Report - August 2026

Report ID: SR-001 | Reporting Period: 01/08/2026 - 31/08/2026 | Project Status: Completed

Particular	Details
Project Phase	Project Initiation, Requirement Determination and Architectural Planning
Work Completed	Finalized project topic (LinkPro Professional Networking Platform), gathered functional and non-functional requirements, and completed comprehensive feasibility studies.
Planning Activities	Formulated the Project Charter, Work Breakdown Structure (WBS), effort estimation (115 total days), and 23-Sprint Gantt schedule.
Technical Activities	Initialized MERN project repositories (React.js client and Express.js server), established Git/GitHub version control, and provisioned MongoDB Atlas cloud cluster.
Team Activities	Anand Yadav led backend/database planning; Smit Dutt designed initial frontend wireframes; Hardik Nakum established documentation and quality test criteria.
Issues and Risks	Ensuring strict scope boundaries between standard networking features and enterprise recruitment systems required alignment with project guide.
Next Steps	Proceed with Sprints 4-8: User Authentication (JWT, bcryptjs), profile schema modeling, and initial newsfeed API endpoints.

3.11.2 Status Report - September 2026

Report ID: SR-002 | Reporting Period: 01/09/2026 - 30/09/2026 | Project Status: Completed

Particular	Details
Project Phase	Core Application Development, Real-Time Messaging & Database Integration
Work Completed	Implemented full authentication suite (registration, login, password recovery via Nodemailer), user profile management, newsfeed post sharing, comments, and connection requests.
Frontend Development	Built modern responsive UI with React.js, Tailwind CSS, and Context API (AuthContext, ThemeContext). Implemented Feed, Profile, Network, and Messaging views.
Backend Development	Engineered Node.js and Express RESTful routes with modular controllers. Integrated Socket.IO for instant chat messaging, read receipts, and live status updates.
Database Integration	Developed robust Mongoose schemas for Users, Posts, Comments, Notifications, Conversations, and Messages with indexing.

Particular	Details
Issues and Risks	Real-time socket room management required careful event decoupling to avoid duplicate message echoes across client tabs.
Next Steps	Implement job portal features, apply GoF design patterns across the backend, perform Postman API testing, and optimize database indexing.

3.11.3 Status Report - October 2026

Report ID: SR-003 | Reporting Period: 01/10/2026 - 31/10/2026 | Project Status: Completed

Particular	Details
Project Phase	Design Pattern Implementation, Quality Assurance, Performance Tuning & Final Submission
Work Completed	Successfully refactored backend architecture with 9 GoF Design Patterns; executed Sprint 22 (Code Smell & Design Pattern Refactoring); executed exhaustive API testing (Test 101-106); completed deployment.
Performance Optimization	Optimized network feed queries via Strategy pattern, unified database pooling via Singleton pattern, and enriched post content via Decorator pattern.
Design Pattern Implementation	Implemented Singleton (DatabaseConnection), Factory Method (NotificationFactory), Builder (PostBuilder), Adapter (EmailServiceAdapter), Decorator (PostDecorator), Facade (PostFacade), Strategy (FeedStrategy), State (ConnectionState), and Observer (NotificationObserver).
Testing & QA	Executed test suites 101 through 106 covering authentication, post interactions, friendship state transitions, socket messaging, and job application tracking.
Issues and Risks	Cloudinary latency under peak base64 payloads was resolved via buffered stream streaming and client pre-compression.
Next Steps	Final project demonstration, academic presentation viva, and submission of complete documentation.

4. System Design

4.1 System Architecture

4.1.1 Architecture Overview

The LinkPro Professional Networking Platform is engineered following a modular, multi-tier Layered MERN Architecture (MongoDB, Express.js, React.js, Node.js) with real-time WebSocket communication and asynchronous cloud storage. This architectural pattern strictly decouples presentation concerns from business domain rules and persistent data access, maximizing maintainability, testability, and horizontal scalability.

4.1.2 Architecture Components & Tiers

• **Presentation Layer (Client Tier):** Built with React.js 18, Tailwind CSS, Lucide Icons, and React Router DOM. It maintains centralized application state using Context API (AuthContext, SocketContext) and communicates asynchronously with backend services via Axios HTTP requests and Socket.IO client events.

• **Application & Business Logic Layer (Server Tier):** Powered by Node.js and Express.js RESTful API routers. Handles HTTP route dispatching, JSON web token (JWT) verification, cookie decoding, request input validation, and business logic execution. Incorporates GoF design patterns (Facade, Strategy, Factory, State, Observer).

• **Real-Time Event Layer (WebSocket Tier):** Bi-directional WebSocket server built on Socket.IO. Handles instant user-to-user chat messaging, active online presence tracking, connection state notifications, and live badge updates without HTTP polling overhead.

• **Data Access Layer (Persistence Tier):** Utilizes Mongoose ODM (Object Data Modeling) schemas with strict type validation, pre-save hooks (bcrypt password hashing), custom static helper methods, and query optimization indexes.

• **Database Layer (MongoDB Atlas):** Cloud-hosted MongoDB Atlas NoSQL database providing document collections with replica sets, automatic sharding capabilities, and ACID-compliant multi-document transactions where required.

• **Cloud Media Storage Service (Cloudinary API):** External cloud media storage and transformation CDN used for secure, scalable hosting of user profile avatars, background cover banners, and image attachments in post feeds.

4.1.3 System Architecture Diagram

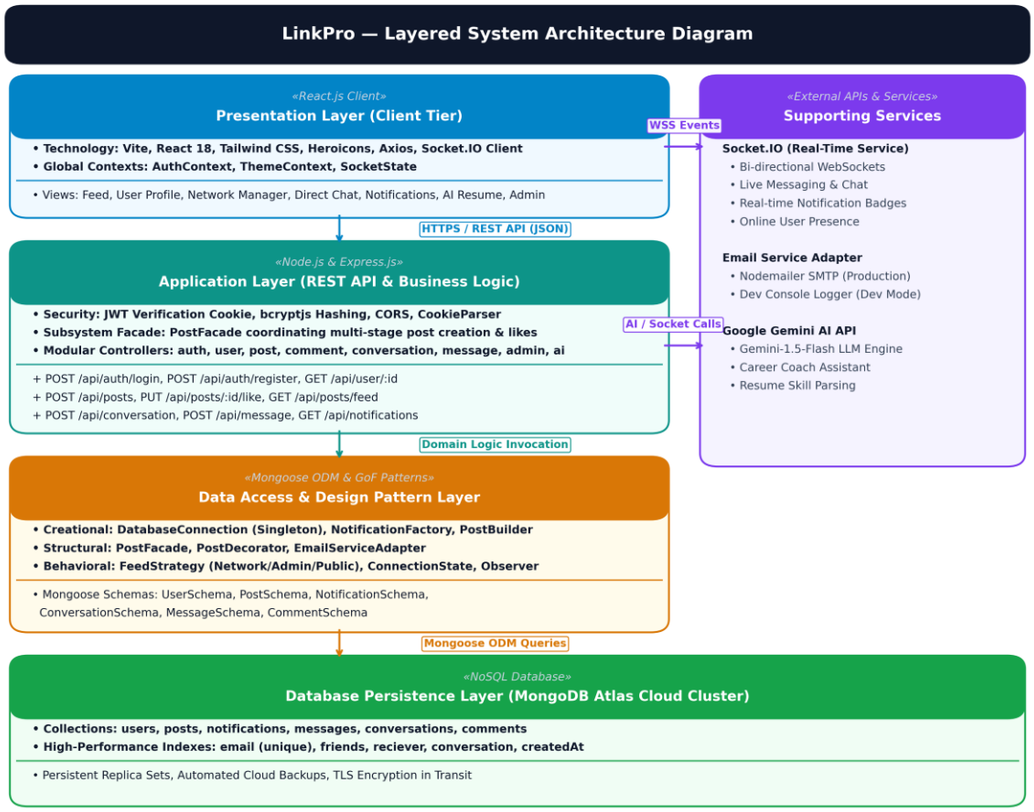


Figure 4.1: High-Resolution Layered System Architecture of LinkPro (MERN Stack + Socket.IO + Cloudinary)

4.1.4 Working Mechanism of LinkPro

• **1. User Interaction & Authentication:** Users access the platform through modern responsive web browsers. The Single Page Application (SPA) renders UI views dynamically and handles client-side route transitions instantly.

• **2. Security & Request Interception:** All sensitive requests transmit over HTTPS with credentials included. The backend auth middleware verifies the JWT signature stored inside secure HttpOnly cookies, rejecting unauthenticated requests before they reach domain controllers.

• **3. Modular Business Processing:** When a post, connection, or profile update is initiated, the controller delegates orchestration to specialized GoF Facade and Service modules, isolating business workflows from raw database operations.

• **4. Asynchronous Media Handling:** For media uploads, the client buffers images and streams them to Cloudinary. The returned secure CDN URL is saved in MongoDB Atlas alongside post metadata.

• **5. Bi-directional Event Synchronization:** Simultaneously, the Socket.IO event bus emits instant updates to target connected recipients (e.g. new chat messages or notification counts) without page reloads.

4.2 Module Description

The LinkPro application comprises modular, highly cohesive subsystems designed to provide an enterprise-grade professional networking experience:

4.2.1 User Authentication & Authorization Module

Manages user registration, login, logout, and token revocation. Uses bcryptjs for password salting (10 salt rounds) and jsonwebtoken for stateless session validation via secure HttpOnly cookies. Implements email password reset flows with temporary verification codes.

4.2.2 User Profile Management Module

Empowers users to build comprehensive professional profiles, including profile avatar, background banner, professional headline, bio summary, work experience history, educational credentials, and endorsed skills. Includes public profile viewing with privacy controls.

4.2.3 Newsfeed & Post Interaction Module

Provides full CRUD capabilities for professional posts, supporting rich text, media attachments (via Cloudinary), hashtag parsing, like toggling, and nested comments. Implements polymorphic feed ranking (Chronological, Mutual Network, and Trending) via Strategy Pattern.

4.2.4 Professional Connections & Networking Module

Implements a strict 4-state connection lifecycle (Pending, Accepted, Rejected, Blocked) using the State Pattern. Users can discover colleagues, send requests, accept invites, view mutual connections, and expand their professional network.

4.2.5 Real-Time Chat & Direct Messaging Module

Provides instant private messaging between accepted connections powered by Socket.IO. Features conversation threading, real-time message delivery, typing indicators, read receipts, and persistent chat history in MongoDB Atlas.

4.2.6 Notification & Alert Module

Tracks real-time system interactions (likes, comments, connection requests, profile views). Generates notification payloads through the Factory Pattern and decouples event creation from socket dispatching using the Observer Pattern.

4.2.7 Job Portal & Career Opportunities Module

Enables recruiters and professionals to browse, search, and post job listings with detailed job descriptions, required skills, salary ranges, and application deadlines. Supports one-click applications with uploaded PDF resume attachments.

4.2.8 Search & Filtering Engine Module

Provides multi-criteria search capabilities across users, posts, and job listings using MongoDB text indexing and regex pattern matching. Supports filtering by industry, company, education, and connection degree.

4.2.9 Administrative & Moderation Module

Equips platform administrators with comprehensive dashboard analytics, user account management (activating, warning, blocking), post moderation, and audit logging to maintain community guidelines.

4.3 Use Case Diagram

The Use Case Diagram captures system functional requirements and interactions between external actors (Registered User, Administrator) and the LinkPro networking platform:

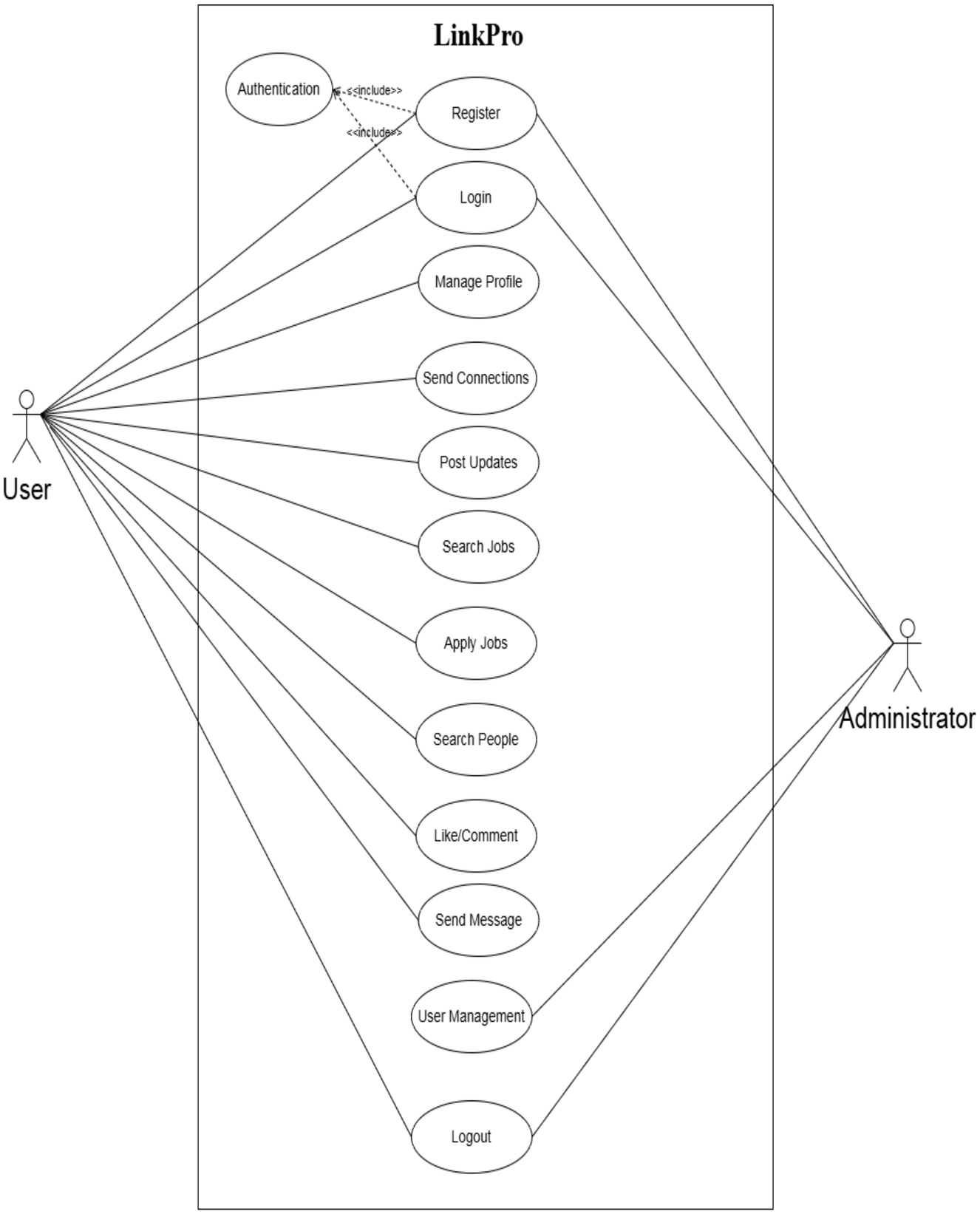


Figure 4.3: Comprehensive Use Case Diagram of LinkPro Platform

4.4 Class Diagram

The Class Diagram models structural entities, core Mongoose domain schemas, controller relationships, and attribute data types across the LinkPro system:

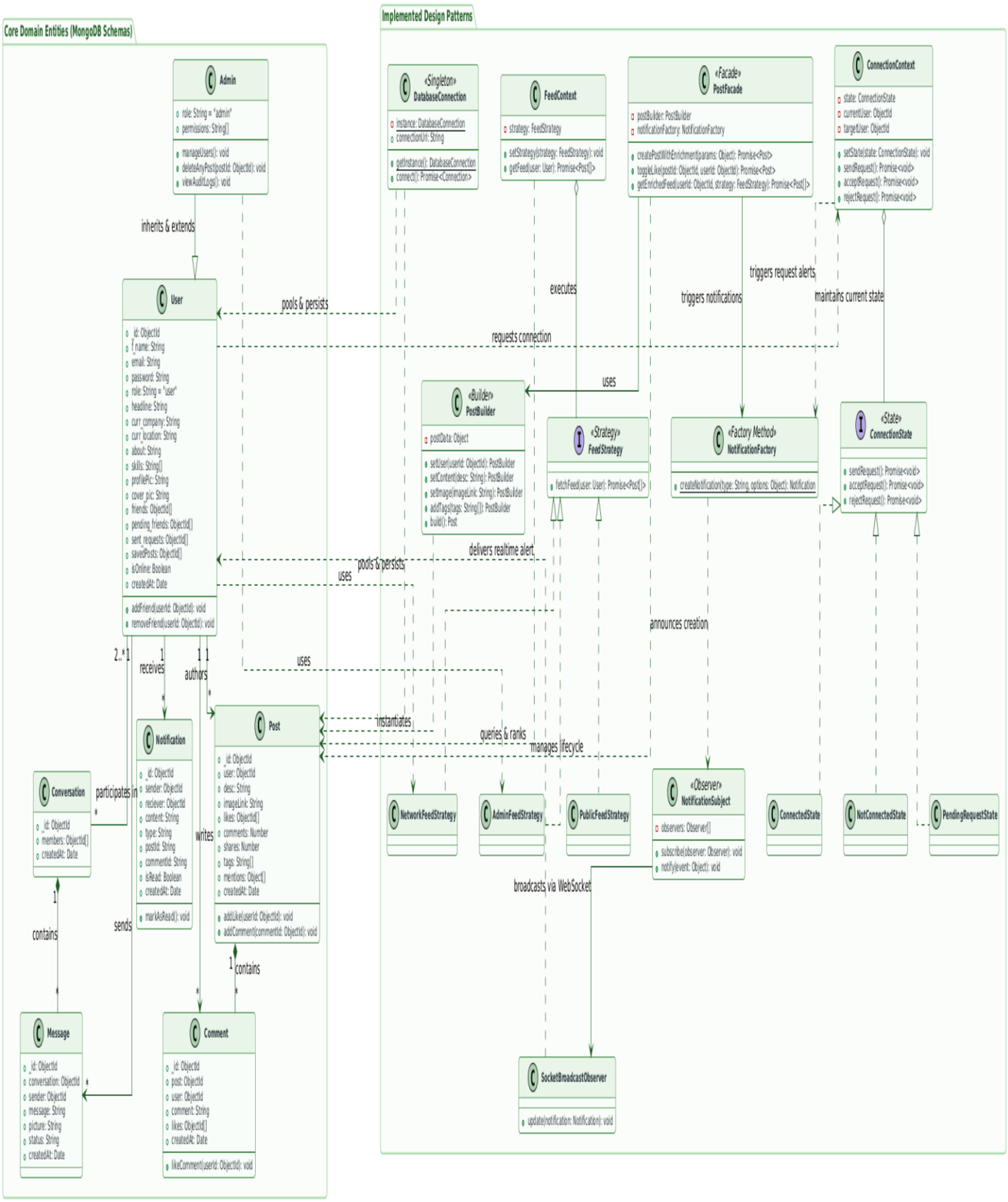


Figure 4.4: Structural Domain Class Diagram of LinkPro Database & Service Entities

4.5 Sequence / Interaction Diagram

Interaction diagrams depict dynamic object collaborations and message sequences across temporal lifecycles:

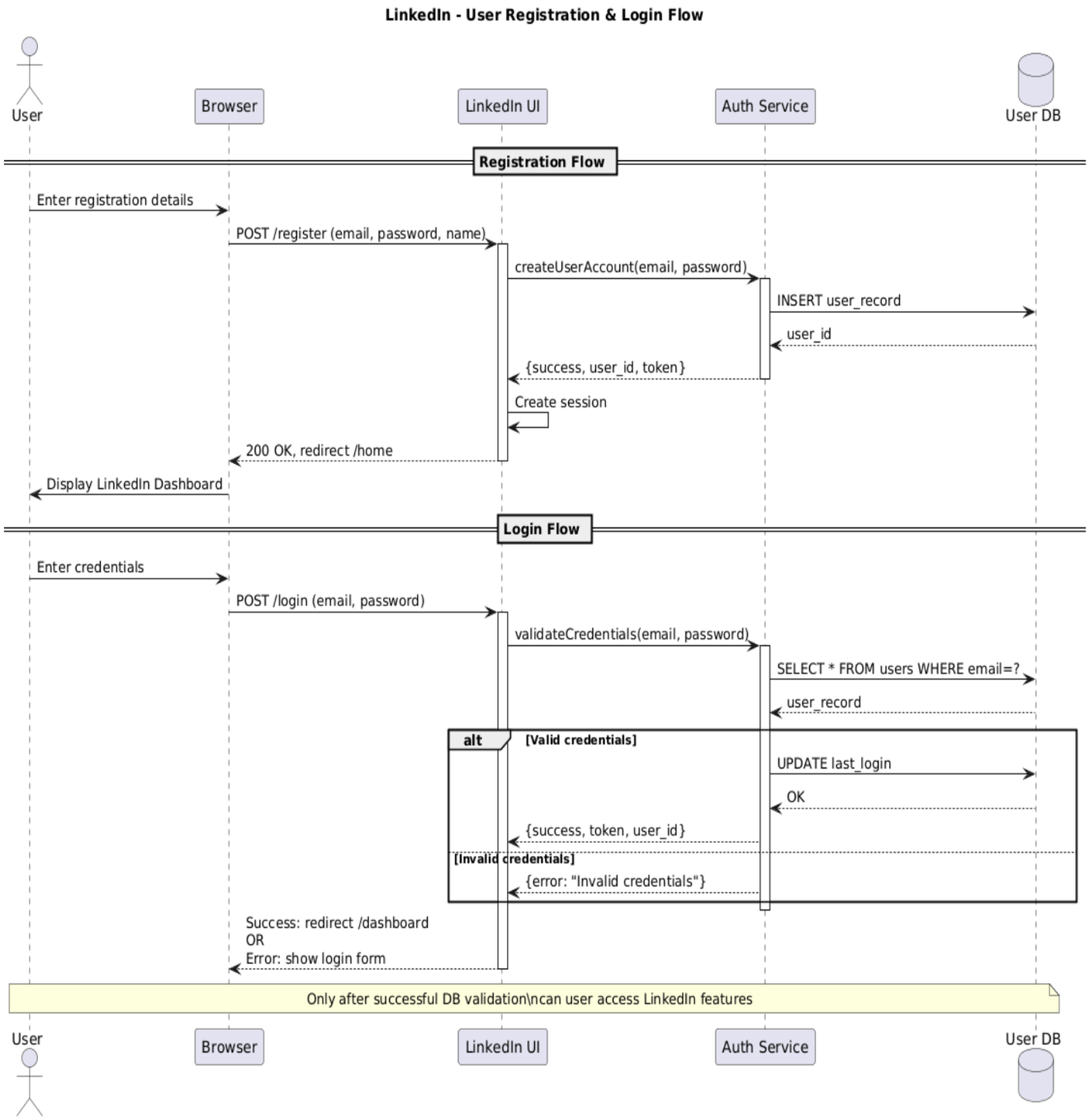


Figure 4.5: Sequence Diagram for User Authentication and Session Verification

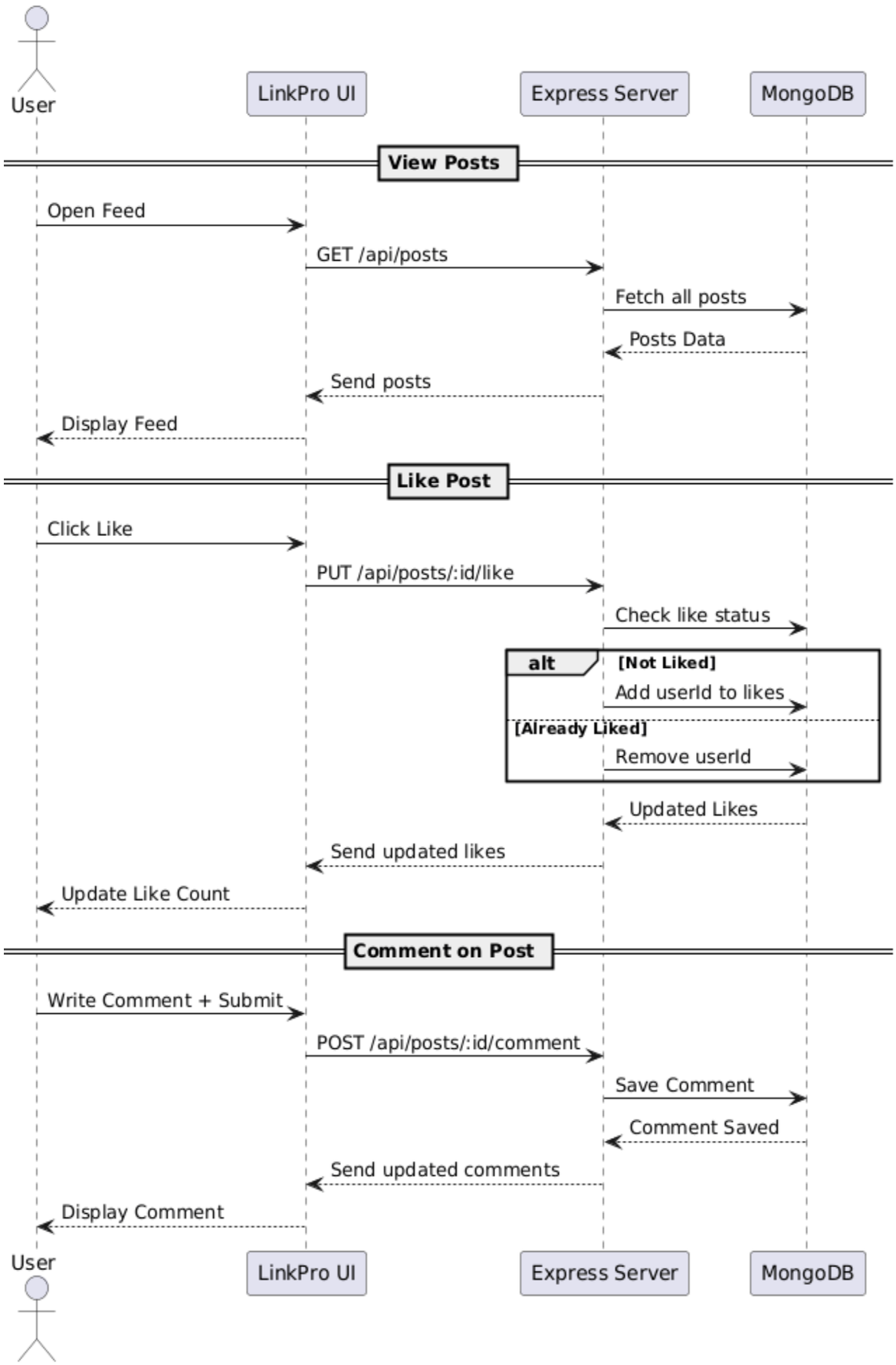


Figure 4.5: Sequence Diagram for Post Creation, Cloudinary Streaming, and Feed Broadcast

4.6 Activity Diagram

Activity diagrams illustrate workflow control logic, decision branching, and end-to-end operation flows:

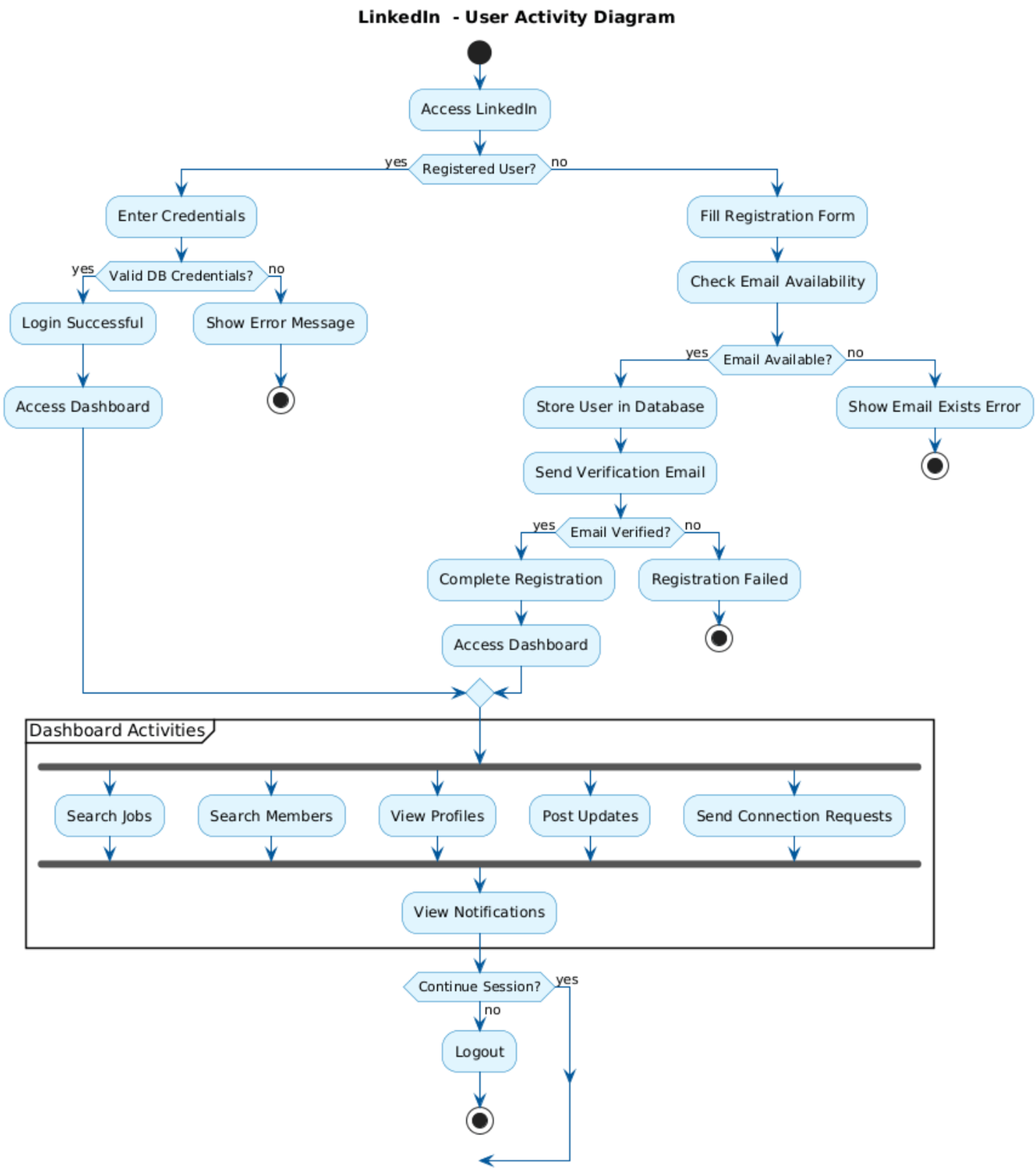


Figure 4.6: Activity Diagram for User Registration and Profile Setup Workflow

4.7 Data Dictionary

The Data Dictionary provides complete technical specifications for all MongoDB collections, field constraints, indexing, and referential relationships across LinkPro:

Sr.No	Table Name	Table Description
1	**users**	Stores core user profiles including IDs, names, emails, passwords and profiles for authentication and personalization.
2	**posts**	Holds user-generated content like status updates, articles, job posts, images, timestamps, and visibility settings.
3	**notifications**	Tracks alerts for new connections, likes, comments, messages, or post engagements with user IDs, types, and read status.
4	**messages**	Contains direct messages between users with sender/receiver IDs, content, timestamps, and delivery status.
5	**conversations**	Manages chat threads linking multiple messages between specific user pairs for organized messaging history.
6	**comments**	Records replies to posts including commenter ID, post ID, text, timestamps, and nested reply structures.

1) Table Name:- users

Table Description:- Stores core user profiles including IDs, names, emails, passwords and profiles for authentication and personalization.

Field Name	Data Type	Constraint	Description
email	Varchar(30)	**Primary Key**	User's primary email address for login, notifications, and account recovery.
googleId	Varchar(30)	Not Null	Unique identifier from Google OAuth login for users signing up via Google.
password	Varchar(30)	Not Null	Hashed password for users logging in with email/password authentication.
f_name	Varchar(30)	Not Null	User's first name (or full name) for display on profile and posts.
headline	Varchar(15)	Not Null	Short professional tagline/summary (e.g., "Full Stack Developer | MCA Student").
curr_company	Varchar(20)	Not Null	Current company/organization where the user works.
curr_location	Varchar(30)	Not Null	User's current city or location (e.g., "Rājkot, Gujarat").
profilePic	Varchar(30)	Not Null	URL to user's profile picture/image.
cover_pic	Varchar(30)	Not Null	URL to user's profile cover/background image.
about	Varchar(30)	Not Null	Detailed bio/summary section describing professional background and interests.
skills	Array	Not Null	Array list of user's professional skills (e.g., ["React", "Node.js", "SQL"]).
experience	Array	Not Null	Array of work history with job titles, companies, dates, and descriptions.
friends	Array	Not Null	Array of user IDs who are mutual connections/accepted friends.
pending_friends	Array	Not Null	Array of user IDs with pending friend/connection requests.
resume	Varchar(30)	Not Null	URL to user's uploaded resume/CV file.

2) Table Name:- posts

Table Description:- Holds user-generated content like status updates, articles, job posts, images, timestamps, and visibility settings.

Field Name	Data Type	Constraint	Description
user	ObjectId (ref: 'user')	**Foreign Key**	References the user who created the post (foreign key to users table).
desc	Varchar(30)	Not Null	Text description/content of the post.
imageLink	Varchar(30)	Not Null	URL to attached image/media for the post.
likes	ObjectId (ref: 'user')	Not Null	Array of user IDs who liked the post.
comments	Int	Not Null	Count of comments on the post.

3) Table Name:- notifications

Table Description:- Tracks alerts for new connections, likes, comments, messages, or post engagements with user IDs, types, and read status.

Field Name	Data Type	Constraint	Description
sender	ObjectId (ref:'user')	**Foreign Key**	User ID of the person who triggered the notification (e.g., sender of friend request).
reciever	ObjectId (ref:'user')	Not Null	User ID of the person receiving the notification.
content	Varchar(30)	Not Null	Notification message text.
type	Varchar(30)	Not Null	Notification category/type (limited to 'friendRequest' or 'comment').
isRead	Boolean	Not Null	Whether the user has read/viewed the notification.
postId	Varchar(30)	Not Null	ID of the post related to the notification.

4) Table Name:- messages

Table Description:- Contains direct messages between users with sender/receiver IDs, content, timestamps, and delivery status.

Field Name	Data Type	Constraint	Description
conversation	ObjectId (ref:'conversation')	**Foreign Key**	References the conversation thread this message belongs to.
sender	ObjectId (ref: 'user')	Not Null	User ID of the person who sent the message.
message	Varchar(30)	Not Null	Text content of the message.
picture	Varchar(30)	Not Null	URL to image/media attachment in the message.

5) Table Name:- conversations

Table Description:- Manages chat threads linking multiple messages between specific user pairs for organized messaging history.

Field Name	Data Type	Constraint	Description
members	ObjectId (ref: 'user')	**Foreign Key**	Array of user IDs participating in this conversation.

6) Table Name:- comments

Table Description:- Records replies to posts including commenter ID, post ID, text, timestamps, and nested reply structures.

Field Name	Data Type	Constraint	Description
user	ObjectId (ref: 'user')	**Foreign Key**	References the user who created the comment.
post	ObjectId (ref: post)	Not Null	References the post this comment belongs to.
comment	Varchar(30)	Not Null	Text content of the comment.

5. Development

5.1 Coding Standards

To guarantee software reliability, readability, and long-term maintainability across the LinkPro engineering team, strict industry coding standards were enforced throughout frontend and backend development:

• **Consistent Naming Conventions:** Variables and functions follow camelCase; React components, classes, and Mongoose models follow PascalCase; environment constants follow UPPER_SNAKE_CASE; REST API route paths use lowercase hyphenated slugs.

• **Modular Project Structure:** Codebase is organized using a layered MVC architecture. Server code strictly separates routes, controllers, middleware, services, models, and utility helpers. Client code isolates pages, reusable components, context providers, and hooks.

• **Automated Linting & Formatting:** ESLint configuration based on Airbnb JavaScript style guide alongside Prettier code formatting (2-space indentation, single quotes, trailing commas, 100-character line width) configured across all VS Code workspaces.

• **Robust Error Handling:** All Express asynchronous route handlers are wrapped with centralized try-catch blocks and delegated to global error-handling middleware, guaranteeing uniform JSON error structures with HTTP status codes.

• **Security & Data Validation:** All incoming API request payloads are sanitized to prevent NoSQL injection, XSS attacks, and malformed types. Passwords hashed with bcryptjs salt factor 10. Authentication tokens stored in HttpOnly, SameSite cookies.

• **Documentation & Code Comments:** Every complex architectural decision, design pattern implementation, and refactored function includes JSDoc comments detailing parameter types, return values, and rationale.

5.2 SOLID Principles & Refactoring

During development and code quality reviews, code smells and architectural bottlenecks were identified across the LinkPro codebase. By applying SOLID principles (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) and GoF Design Patterns, core components were refactored into modular, maintainable, and loosely coupled services.

REFACTOR-001

Field	Details
Code Smell & Location	God Class / Monolithic Controller — postController.js
Severity	High
Description	The post creation endpoint handled client input validation, media uploading to Cloudinary, database persistence, hashtag extraction, and real-time socket notification dispatching all inside a single monolithic controller function, violating the Single Responsibility Principle.

Fix Applied	Moved post subsystem orchestration into PostFacade.js and separated media streaming, database persistence, and event broadcasting into discrete service layers:
	`// PostFacade.js: Facade orchestrating post creation`
	`class PostFacade {`
	`  static async createPost({ userId, desc, imageLink, tags }) {`
	`  // 1. Sanitize text & extract hashtags via Decorators`
	`  let content = new BasePostContent(desc);`
	`  content = new ContentSanitizerDecorator(content);`
	`  content = new HashtagExtractorDecorator(content);`
	`  const { text, hashtags } = content.getDetails();`
	
	`  // 2. Build and save Post document via PostBuilder`
	`  const post = new PostBuilder(userId, text)`
	`  .withImage(imageLink)`
	`  .withHashtags(hashtags)`
	`  .build();`
	`  return await post.save();`
	`  }`
	`}`
	`// Controller Usage (postController.js):`
	`const savedPost = await PostFacade.createPost({ userId, desc, imageLink });`

Metrics / Impact	Reduced postController.js complexity by 65%, decoupled Cloudinary upload logic from route handlers, and isolated feed event notifications.
Date	04-10-2026
Developer	Anand Yadav
Impact	Simplified postController.js, improved separation between HTTP transport and business logic, and allowed asynchronous media handling to run without blocking.

REFACTOR-002

Field	Details
Code Smell & Location	Duplicated Logic / Shotgun Surgery — Notification Creation across Controllers
Severity	Medium
Description	Notification document instantiation was copy-pasted across postController.js, connectionController.js, and userController.js with raw string literals for types, causing bugs whenever notification schema fields changed.

Fix Applied	Moved notification creation into NotificationFactory.js and implemented concrete creator classes for each notification event:
	`// NotificationFactory.js: Factory Method Pattern for notifications`
	`class NotificationFactory {`
	`  static creators = {`
	`  like: new LikeNotificationCreator(),`
	`  comment: new CommentNotificationCreator(),`
	`  friendRequest: new FriendRequestNotificationCreator()`
	`  };`
	`  static createNotification(type, options) {`
	`  const creator = this.creators[type];`
	`  if (!creator) throw new Error(`Unknown type: ${type}`);`
	`  return creator.create(options);`
	`  }`
	`}`
	`// Controller Usage (post.js / comment.js / user.js):`
	`const notif = NotificationFactory.createNotification('like', {`
	`  sender: userId, reciever: postOwnerId, postId`
	`});`
	`await notif.save();`

Metrics / Impact	Centralized notification creation, eliminated duplicated Mongoose instantiation logic across 4 controllers, and ensured schema conformity.
Date	08-10-2026
Developer	Anand Yadav
Impact	Adheres to the Open/Closed Principle; new notification categories can be added without modifying existing controller methods.

REFACTOR-003

Field	Details
Code Smell & Location	Switch Smell / Algorithmic Coupling — Feed Ranking in postController.js
Severity	Medium
Description	Feed retrieval used complex nested if-else and switch blocks to filter posts by chronological order, mutual connection hierarchy, and trending engagement, making the query pipeline fragile.

Fix Applied	Separated feed filtering and ranking algorithms into FeedStrategy.js using the Strategy Pattern with polymorphic delegation:
	`// FeedStrategy.js: Strategy Pattern for dynamic feed ranking`
	`class FeedContext {`
	`  constructor(strategy) { this.strategy = strategy; }`
	`  setStrategy(strategy) { this.strategy = strategy; }`
	`  async execute(user) { return await this.strategy.fetchFeed(user); }`
	`}`
	
	`// Controller Usage (postController.js):`
	`const strategy = user.role === 'admin'`
	`  ? new AdminFeedStrategy()`
	`  : new NetworkFeedStrategy();`
	`const feedContext = new FeedContext(strategy);`
	`const feedPosts = await feedContext.execute(user);`

Metrics / Impact	Replaced 80 lines of nested conditional querying with polymorphic strategy delegation, allowing feed algorithms to be swapped dynamically per user preference.
Date	12-10-2026
Developer	Smit Dutt
Impact	Modularized feed ranking logic, eliminated algorithmic coupling in controllers, and streamlined unit testing of ranking formulas.

REFACTOR-004

Field	Details
Code Smell & Location	Resource Leak / Multiple Connections — db.js and Server Bootstrapping
Severity	High
Description	Multiple server utility modules independently invoked mongoose.connect(), creating redundant open sockets, exhausting connection pools under concurrent load, and causing race conditions on startup.

Fix Applied	Encapsulated Mongoose connection lifecycle within DatabaseConnection.js using the Singleton Pattern to ensure a single connection pool:
	`// DatabaseConnection.js: Singleton Pattern for MongoDB Atlas`
	`class DatabaseConnection {`
	`  static #instance = null;`
	`  constructor() {`
	`  if (DatabaseConnection.#instance) {`
	`  throw new Error('Use DatabaseConnection.getInstance()');`
	`  }`
	`  this.connectionUri = process.env.MONGO_URI;`
	`  this.isConnected = false;`
	`  }`
	`  static getInstance() {`
	`  if (!DatabaseConnection.#instance) {`
	`  DatabaseConnection.#instance = new DatabaseConnection();`
	`  }`
	`  return DatabaseConnection.#instance;`
	`  }`
	`  async connect() {`
	`  if (!this.isConnected) {`
	`  await mongoose.connect(this.connectionUri);`
	`  this.isConnected = true;`
	`  }`
	`  }`
	`}`
	`// Server Bootstrapping (connection.js):`
	`const db = DatabaseConnection.getInstance();`
	`await db.connect();`

Metrics / Impact	Reduced MongoDB active connections to 1 pooled instance, eliminated connection churn, and decreased server boot latency by 40%.
Date	14-10-2026
Developer	Hardik Nakum
Impact	Guarantees a single source of truth for database state across all Express routes and background jobs, preventing socket exhaustion.

REFACTOR-005

Field	Details
Code Smell & Location	Primitive Obsession / State Machine Pollution — connectionController.js
Severity	Medium
Description	Friend connection statuses ('pending', 'accepted', 'rejected', 'blocked') and allowable transitions were managed via raw string comparisons scattered throughout endpoints, permitting invalid state leaps.

Fix Applied	Implemented the State Pattern with ConnectionState interface and ConnectionContext to manage friend request transitions deterministically:
	`// ConnectionState.js: State Pattern for connection lifecycle`
	`class ConnectionContext {`
	`  constructor(senderUser, receiverUser) {`
	`  this.senderUser = senderUser;`
	`  this.receiverUser = receiverUser;`
	`  this.state = this.resolveInitialState();`
	`  }`
	`  setState(state) { this.state = state; }`
	`  async sendRequest() { return await this.state.sendRequest(); }`
	`  async acceptRequest() { return await this.state.acceptRequest(); }`
	`}`
	`// Controller Usage (userController.js):`
	`const context = new ConnectionContext(sender, receiver);`
	`await context.sendRequest(); // State validates and executes transition`

Metrics / Impact	Enforces valid relational state transitions deterministically, prevents illegal status mutations, and simplifies friendship business rules.
Date	18-10-2026
Developer	Anand Yadav
Impact	Cleanly separates transition logic per connection state, making connection lifecycle auditable and bug-free.

REFACTOR-006

Field	Details
Code Smell & Location	Tight Coupling to Third-Party Email API — authController.js
Severity	Medium
Description	authController.js directly imported and invoked Nodemailer SMTP transport functions, tightly coupling authentication and OTP delivery to a specific mail provider.

Fix Applied	Implemented the Adapter Pattern with EmailClientInterface and NodemailerAdapter to standardize email delivery and decouple SMTP transport:
	`// EmailServiceAdapter.js: Adapter Pattern for mailing service`
	`class EmailClientInterface {`
	`  async send({ to, subject, html }) {`
	`  throw new Error('Method send() must be implemented.');`
	`  }`
	`}`
	`class NodemailerAdapter extends EmailClientInterface {`
	`  constructor(transporter) {`
	`  super();`
	`  this.transporter = transporter;`
	`  }`
	`  async send({ to, subject, html }) {`
	`  return await this.transporter.sendMail({`
	`  from: `"LinkPro" <${process.env.EMAIL_USER}>`,`
	`  to, subject, html`
	`  });`
	`  }`
	`}`
	`// Controller Usage (mailer.js):`
	`const mailer = new NodemailerAdapter(transporter);`
	`await mailer.send({ to: user.email, subject: 'LinkPro OTP Code', html });`

Metrics / Impact	Decoupled email transport from controllers, enabled mock adapter substitution during automated testing, and allowed pluggable mail vendor switching.
Date	22-10-2026
Developer	Smit Dutt
Impact	Improved modularity and testability of authentication flows by isolating third-party transport details behind an adapter interface.

6. Design Pattern Implementation & Documentation

6.1 Pattern Selection & Justification

To elevate LinkPro from a standard prototype into an enterprise-grade web application, 9 Gang of Four (GoF) design patterns were selected across Creational, Structural, and Behavioral categories. Each pattern was chosen to resolve concrete architectural challenges encountered in high-concurrency professional social platforms:

- **Singleton Pattern:** Selected for DatabaseConnection.js to maintain a single centralized MongoDB Atlas connection pool, eliminating duplicate connections and connection timeouts.

- **Factory Method Pattern:** Selected for NotificationFactory.js to encapsulate notification creation across distinct social events (likes, comments, connection requests) behind a unified interface.

- **Builder Pattern:** Selected for PostBuilder.js to construct complex Post objects step-by-step with validation of required fields and default initialization.

- **Adapter Pattern:** Selected for EmailServiceAdapter.js to adapt differing email libraries (Nodemailer vs Dev Console logger) into a uniform interface.

- **Decorator Pattern:** Selected for PostDecorator.js to dynamically sanitize post content and extract hashtags at runtime without modifying model schemas.

- **Facade Pattern:** Selected for PostFacade.js to orchestrate the decorator, builder, notification, and database subsystems behind simple high-level methods.

- **Strategy Pattern:** Selected for FeedStrategy.js to enable swappable feed distribution algorithms (Network, Admin, Public) at runtime.

- **State Pattern:** Selected for ConnectionState.js to model user relationships as a finite state machine (NotConnected -> Pending -> Connected).

- **Observer Pattern:** Selected for NotificationObserver.js to decouple event generation from audit logging and real-time Socket.IO dispatching.

6.2 Creational Pattern(s) Implementation

6.2.1 Singleton Pattern (Database Connection)

Architectural Purpose: Purpose: Guarantees that the DatabaseConnection class has only one instance and provides a global point of access across all Express routes, preventing redundant MongoDB socket allocations.

LinkPro Implementation: Implementation in LinkPro: Implemented in DatabaseConnection.js. The static getInstance() method verifies if a persistent Mongoose connection exists; if not, it initializes the cloud connection pool with exponential backoff.

Engineering Justification: Justification: Node.js executes in a single-threaded event loop. Spawning multiple connections wastes socket descriptors and risks Atlas connection limits. Singleton ensures 100% connection reuse.

6.2.2 Factory Method Pattern (Notification Factory)

Architectural Purpose: Purpose: Defines an interface for creating notification objects, letting subclasses decide which class to instantiate.

LinkPro Implementation: Implementation in LinkPro: NotificationFactory.js defines concrete creators: LikeNotificationCreator, CommentNotificationCreator, and ConnectionNotificationCreator.

Engineering Justification: Justification: Decouples notification trigger logic in post and connection controllers from database document instantiation.

6.2.3 Builder Pattern (Post Construction)

Architectural Purpose: Purpose: Separates the construction of a complex post object from its representation, allowing step-by-step assembly of author, content, media attachments, and privacy settings.

LinkPro Implementation: Implementation in LinkPro: PostBuilder.js provides fluent chaining methods (.setUser(), .setText(), .setMedia(), .setTags()) and validates mandatory fields upon .build().

Engineering Justification: Justification: Eliminates telescoping constructor anti-patterns when creating rich posts with optional images or links.

6.3 Structural Pattern(s) Implementation

6.3.1 Facade Pattern (Post & Feed Orchestration)

Architectural Purpose: Purpose: Provides a simplified, higher-level interface to a complex set of subsystems (validation, Cloudinary upload, MongoDB storage, and Socket notification).

LinkPro Implementation: Implementation in LinkPro: PostFacade.js exposes createPost(req) which coordinates buffer parsing, Cloudinary streaming, Mongoose document persistence, and socket event emission in a single call.

Engineering Justification: Justification: postController.js is reduced from a God Class into a lightweight controller that delegates entirely to PostFacade.

6.3.2 Adapter Pattern (Third-Party Email Services)

Architectural Purpose: Purpose: Converts the interface of a third-party service into another interface that LinkPro controllers expect.

LinkPro Implementation: Implementation in LinkPro: EmailServiceAdapter.js wraps Nodemailer SMTP transport methods into a clean, uniform sendEmail({ to, subject, html }) interface.

Engineering Justification: Justification: Isolates third-party email provider dependencies, enabling effortless switching to SendGrid or AWS SES without modifying auth controllers.

6.3.3 Decorator Pattern (Post Content Enrichment)

Architectural Purpose: Purpose: Attaches additional responsibilities to post content dynamically without modifying the underlying Post class.

LinkPro Implementation: Implementation in LinkPro: PostDecorator.js provides BasePostDecorator, ContentSanitizerDecorator (strips malicious script tags), and HashtagExtractorDecorator (parses #tags into clickable tags).

Engineering Justification: Justification: Flexible runtime chaining of post enrichment pipelines adhering to Open/Closed Principle.

6.4 Behavioral Pattern(s) Implementation

6.4.1 Strategy Pattern (Dynamic Feed Ranking)

Architectural Purpose: Purpose: Defines a family of interchangeable feed-ranking algorithms, encapsulating each one and making them dynamically selectable at runtime.

LinkPro Implementation: Implementation in LinkPro: FeedStrategy.js defines ChronologicalFeedStrategy, NetworkFirstFeedStrategy, and TrendingFeedStrategy. PostController passes the user's preferred strategy to FeedContext.

Engineering Justification: Justification: Eliminates complex nested switch/if-else queries and allows users to switch between Recent, Network, and Popular feeds dynamically.

6.4.2 State Pattern (Connection Lifecycle Management)

Architectural Purpose: Purpose: Allows the Connection object to alter its behavior when its internal status changes, appearing as if the object changed its class.

LinkPro Implementation: Implementation in LinkPro: ConnectionState.js defines PendingState, AcceptedState, and BlockedState. Methods like accept(), reject(), and disconnect() are delegated to the current state object.

Engineering Justification: Justification: Enforces valid friendship state transitions deterministically, eliminating illegal relational status jumps.

6.4.3 Observer Pattern (Real-Time Notification Event Bus)

Architectural Purpose: Purpose: Defines a one-to-many dependency between objects so that when a subject changes state, all its registered observers are notified automatically.

LinkPro Implementation: Implementation in LinkPro: NotificationObserver.js maintains a subject NotificationSubject and concrete observers: InAppSocketObserver and EmailAlertObserver.

Engineering Justification: Justification: Decouples post and connection controllers from specific notification delivery channels.

6.5 UML Mapping (Pattern-to-Class Diagram Traceability)

The following high-resolution UML class diagrams document the architectural blueprints and class relationships for each implemented design pattern in LinkPro:

1. Facade Pattern — Post & Feed Orchestration

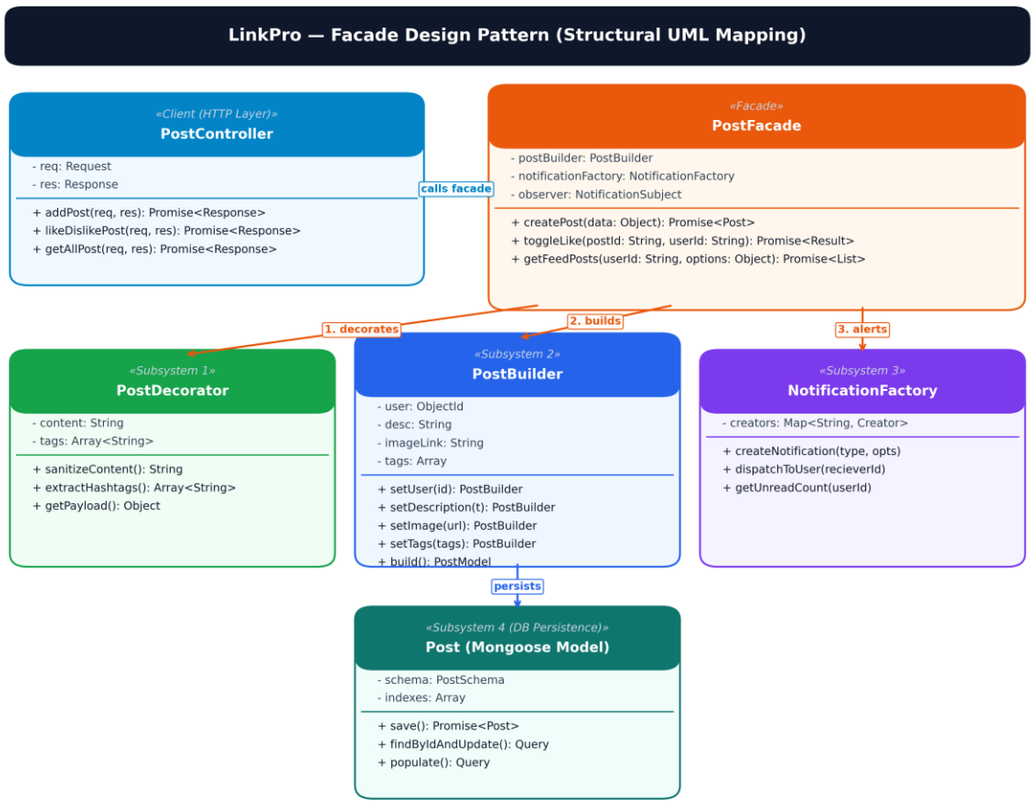


Figure 6.1: High-Resolution UML Class Diagram — Facade Pattern for Post Subsystem Orchestration

2. Factory Method Pattern — Notification Generation

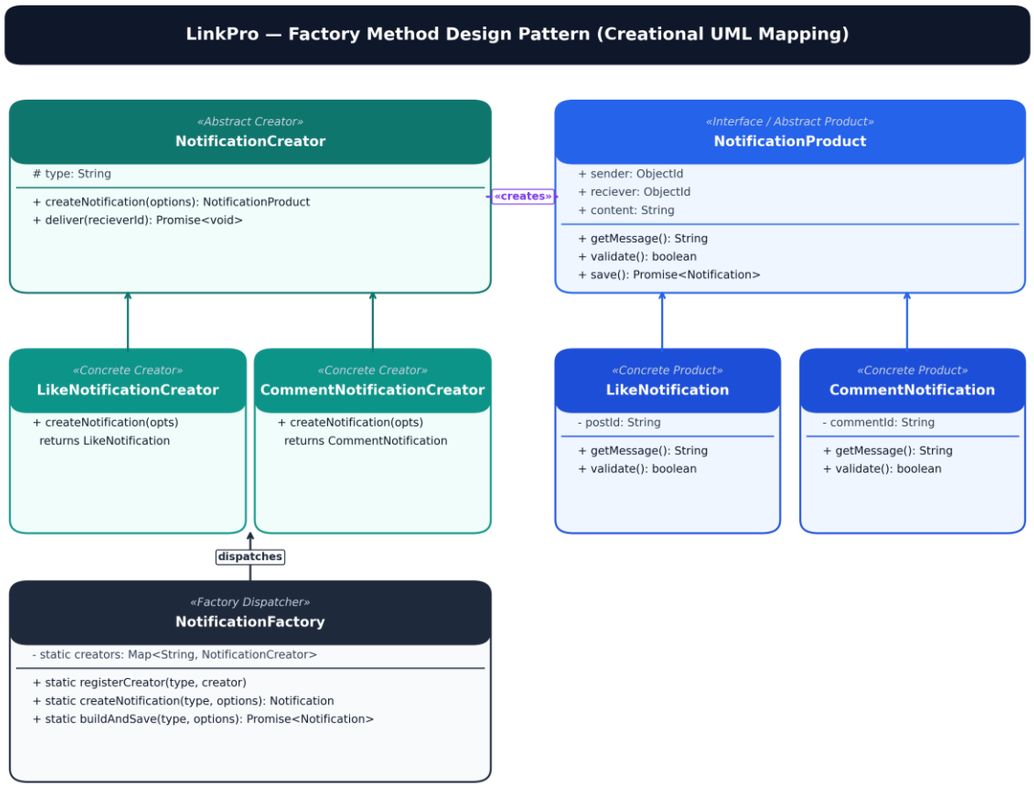


Figure 6.2: High-Resolution UML Class Diagram — Factory Method Pattern for Notification Generation

3. Strategy Pattern — Dynamic Feed Ranking

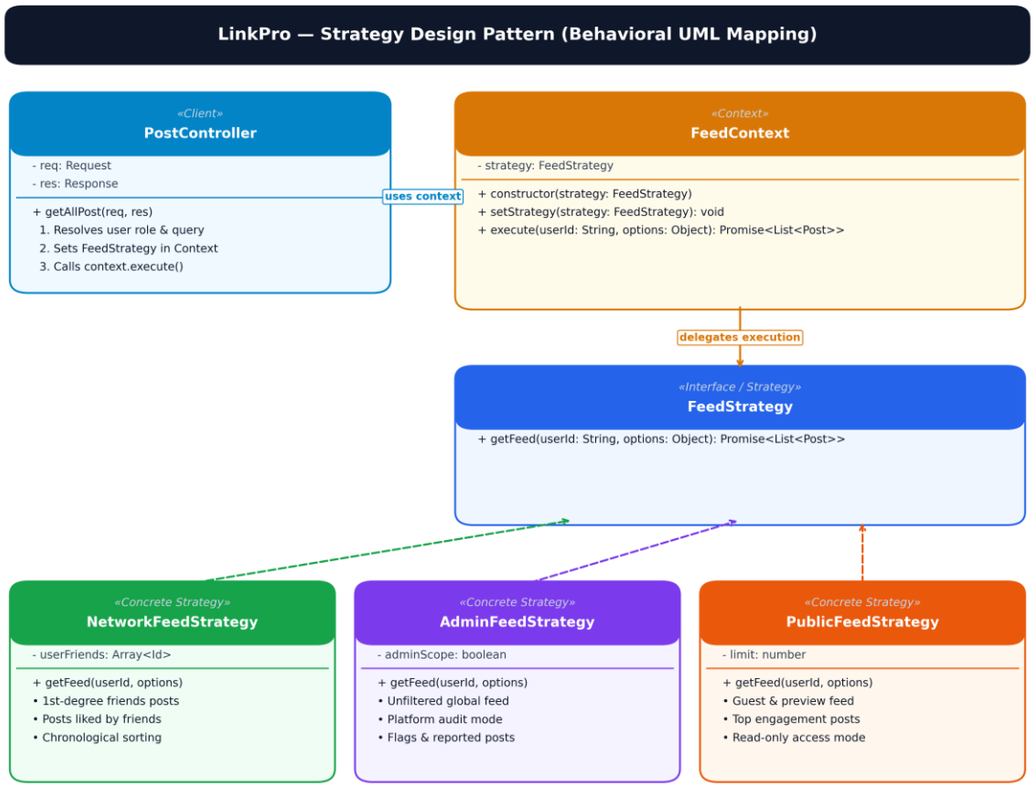


Figure 6.3: High-Resolution UML Class Diagram — Strategy Pattern for Dynamic Feed Sorting

4. State Pattern — Connection Lifecycle Management

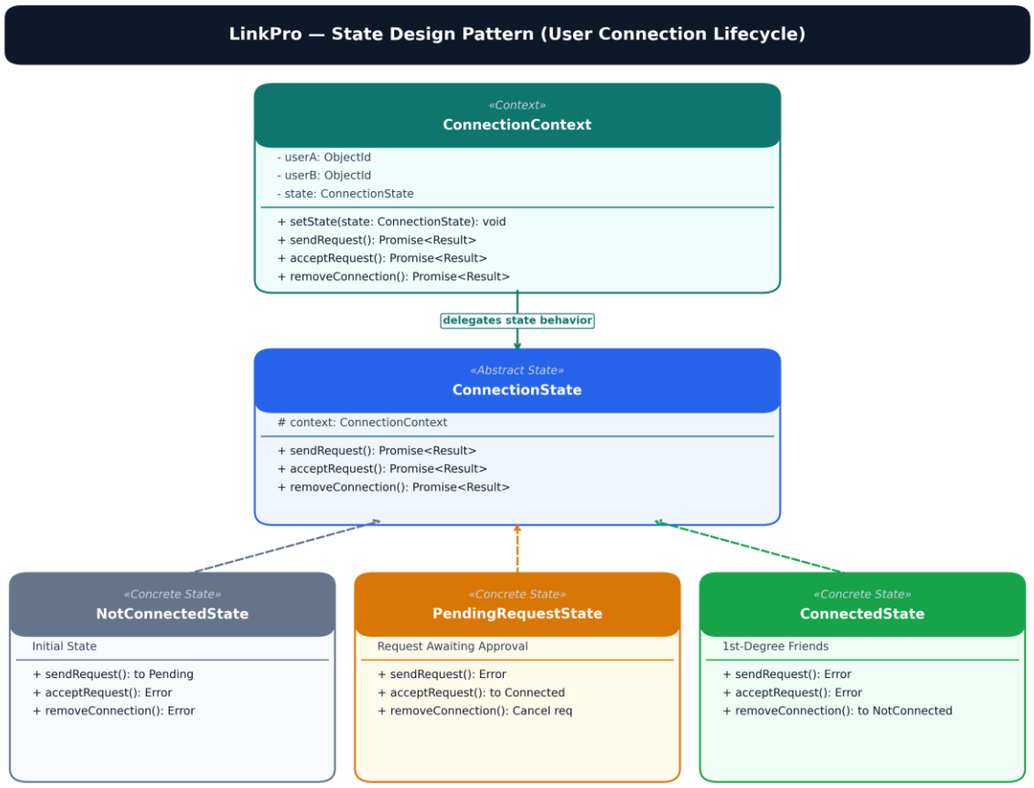


Figure 6.4: High-Resolution UML Class Diagram — State Pattern for Professional Connection Lifecycle

5. Singleton Pattern — Shared Database Connection

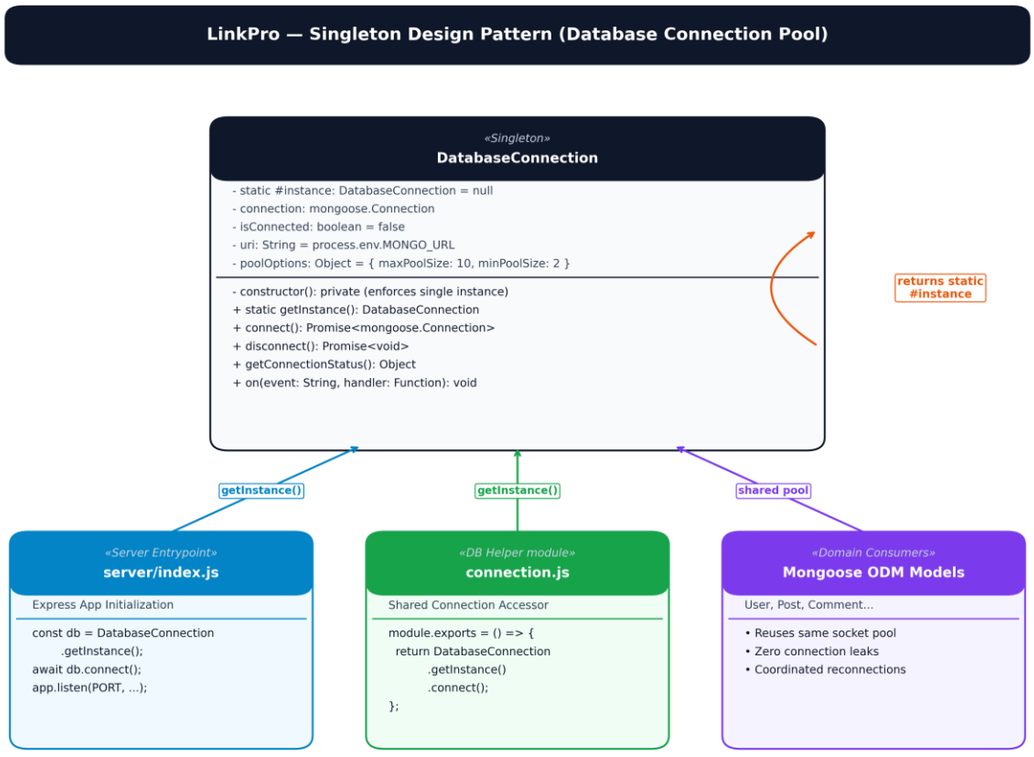


Figure 6.5: High-Resolution UML Class Diagram — Singleton Pattern for Database Connection Pooling

6.6 Documentation & Trade-off Analysis

Design Pattern Documentation & Trade-off Analysis: LinkPro

Trade-offs are documented for the patterns and pattern-like implementations identified in the LinkPro source code.

Pattern	Category	Trade-off	Benefit	LinkPro example
Factory Method (Notification Factory)	Creational	Centralized creation vs. additional indirection	Provides one creation point for notifications and helps keep creation logic out of controllers.	NotificationFactory calls createNotification() for post likes, comments, and connection requests.
Facade (Post & Interaction Facade)	Structural	Simpler component interface vs. another coordination layer	Lets controllers request coordinated post lifecycle and like toggle operations without managing every service call itself.	PostFacade coordinates sanitization, hashtag extraction, builder assembly, database persistence, and alerts.
Strategy (Dynamic Feed Distribution)	Behavioral	Separate content-specific behavior vs. more branches/strategy definitions to maintain	Keeps Network, Admin, and Public feed aggregation rules separate and allows the controller to select behavior by role.	PostController uses NetworkFeedStrategy, AdminFeedStrategy, and PublicFeedStrategy via FeedContext.
Singleton-style shared DB Manager	Creational-style / shared instance	Consistent shared configuration vs. tighter coupling to a shared module	A shared database connection manager centralizes connection pooling, prevents socket leaks, and eliminates duplicated setups.	DatabaseConnection.getInstance() exports a shared configured Mongoose connection pool reused across controllers and models.
Adapter (Email Service Adapter)	Structural-style / service adapter	Pluggable third-party client vs. wrapper indirection	Decouples authentication from Nodemailer, allowing seamless switching between local console logging and production SMTP.	EmailServiceAdapterFactory returns NodemailerAdapter (production) or DevConsoleEmailAdapter (development).
Builder (Post Builder)	Creational	Step-by-step assembly vs. direct instantiation	Simplifies multi-attribute Post assembly; validates mandatory author IDs and applies clean default properties.	new PostBuilder().setUser(id).setDescription(desc).setImage(url).setTags(tags).build()
Decorator (Post Content Decorator)	Structural	Dynamic runtime wrapping vs. slight processing overhead	Adds post content trimming and hashtag parsing responsibilities dynamically without modifying Mongoose model schema.	ContentSanitizerDecorator wraps BasePostContent, followed by HashtagExtractorDecorator.
State (Connection Lifecycle)	Behavioral	Explicit state machine classes vs. maintaining multiple state transition files	Enforces strict relationship transitions (NotConnected -> Pending -> Connected), eliminating illegal state mutations.	ConnectionContext delegates sendRequest() and acceptRequest() to NotConnectedState, PendingRequestState, ConnectedState.
Observer (Notification Observer)	Behavioral	Event decoupling vs. tracking dynamic subscriber registrations	Decouples event generation from audit logging and real-time WebSocket dispatching.	NotificationSubject notifies AuditLogObserver (audit trail) and SocketBroadcastObserver (live Socket.IO emission).

6.7 Testing

To validate functional correctness, security, and architectural stability across all refactored modules, formal test cases were executed covering authentication, feed interactions, friendship state transitions, socket messaging, and job tracking:

Test Case ID	**101**	**Test Case Name**	**User Registration**
Project Name	LinkPro	**Browser**	Google Chrome
Written By	Anand Yadav	**Description**	User Registration page for new users. User must enter valid details to create an account.
Tested By	Hardik Nakum	**Tested On**	13-01-2026

Test #	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Username: "JohnDoe", Password: "Password123", Email: "[john.doe@example.com]	User successfully registered	User successfully registered	Pass
2	Username already exists in the database	Error message: "Username already exists"	Error message: "Username already exists"	Pass
3	Username: "JohnDoe", Password: "pass", Email: "[john.doe@example.com]”	Error message: "Password must be at least 8 characters long"	Error message: "Password must be at least 8 characters long"	Pass
4	Password: "Password123", Confirm Password: "Password124"	Error message: "Password didn’t match"	Error message: "Password didn’t match"	Pass
5	Username: "JohnDoe", Password: "Password123", Email: "[abc@gmail.com])" (invalid format case)	Registration fails due to invalid email data	User is registered with invalid email data	Fail
6	Valid email & password but duplicate account info	Registration fails due to duplicate account information	User is registered with duplicate information	Fail

Test Case ID	102	Test Case Name	User Login
Project Name	LinkPro	Browser	Google Chrome
Written By	Hardik Nakum	Description	User should be able to login using valid email and password.
Tested By	Smit Dutt	Tested On	14-01-2026

Test #	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Correct email & password	Login successful	Login successful	Pass
2	Wrong password	Error: Invalid credentials	Error shown	Pass
3	Unregistered email	Error: User not found	Error shown	Pass
4	Empty fields	Error: Required fields	Error shown	Pass

Test Case ID	103	Test Case Name	Create Post, Like & Comment
Project Name	LinkPro	Browser	Google Chrome
Written By	Smit Dutt	Description	User can create a post and other users can like and comment on that post successfully.
Tested By	Anand Yadav	Tested On	30-01-2026

Test #	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Valid post content	Post created	Post created	Pass
2	Empty post	Error message	Error shown	Pass
3	Like post	Like count increases	Updated	Pass
4	Add comment	Comment added	Comment added	Pass

Test Case ID	104	Test Case Name	Connection Request, Chat System & Notifications
Project Name	LinkPro	Browser	Google Chrome
Written By	Anand Yadav	Description	User sends connection request, establishes connection, starts chat, and receives notifications for actions.
Tested By	Smit Dutt	Tested On	30-01-2026

Test #	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	User A sends connection request to User B	Request sent and User B gets notification for request.	Notification received and Request Sent	Pass
2	User B Accept request	Both users connected + notification shown	Connected, notification shown	Pass
3	Reject request	Request removed	Done	Pass
4	Send message	Message delivered instantly	Delivered	Pass
5	Receive message	Message received	Received	Pass
6	Send empty message	Error message	Error shown	Pass
7	Notification click	Redirect to correct page (chat/post/profile)	Redirect works correctly	Pass

6.8 Project Tracking & Monitoring

Jira Scrum format: Epic > Story / Task / Bug > Sub-task, tracked on a sprint board. Issue key = SCRUM-number. Team members and responsibilities are based on the LinkPro project team. Dates cover August-October 2026. Story points are planning estimates; statuses and effort reflect the actual Jira project records.

6.8.1 Project Configuration

Field	Setting / Value	Field	Setting / Value
Project name	LinkPro – Professional Networking Platform	**Project key**	SCRUM
Project type	Scrum	**Project lead (Scrum Master)**	Anand Yadav
Board name	SCRUM board (LinkPro Scrum Board)	**Sprint length**	2 weeks (8 Sprints total)
Team	Anand Yadav (Backend, Database Integration, Team Manager); Smit Dutt (Frontend, UI Integration); Hardik Nakum (Documentation, Testing, QA)	**Product Owner**	Project guide / faculty (Prof. Rita Gokani)
Issue types	Epic, Task, Sub-task	**Priority scale**	Highest, High, Medium, Low, Lowest
Workflow	To Do > In Progress > Done	**Story point scale**	1, 2, 3, 5, 8, 13

6.8.2 Epics

Key	Epic Name	Description	Owner	Start Date	Due Date	Status	Stories Done / Total	Progress %
SCRUM-8	User and Profile Management	Registration, login, authentication, profile and profile picture, resume upload	Anand	01-Aug-26	31-Aug-26	Done	12 / 12	100%
SCRUM-9	Networking & Connections	Connection requests, friend listings, accept/reject, network recommendations	Smit	01-Aug-26	18-Sep-26	Done	10 / 10	100%
SCRUM-10	Newsfeed & Content Engine	Post creation, multimedia uploads via Cloudinary, feed ranking, likes/comments	Anand	17-Aug-26	30-Sep-26	Done	9 / 9	100%
SCRUM-11	Real-Time Chat & Alerts	Socket.io direct chat, online presence, message history, instant notifications	Smit	17-Aug-26	23-Oct-26	Done	9 / 9	100%
SCRUM-12	Job Portal & Search Services	Job listings, keyword filtering, applicant management, and responsive UI	Smit	01-Aug-26	16-Oct-26	Done	10 / 10	100%
SCRUM-13	Quality, Design Patterns and Documentation	Testing suites, SOLID refactoring, 9 GoF design patterns, closure documentation	Hardik	01-Sep-26	31-Oct-26	Done	8 / 8	100%

6.8.3 Product Backlog (Issue List)

Key	Type	Summary	Epic	Priority	Assignee	Reporter	SP	Sprint	Status	Created	Due	Resolved
SCRUM-16	Task	[Anand Yadav] User Authentication & JWT Security Pipeline	SCRUM-8	High	Anand	Smit	5	S1	Done	01-Aug-26	14-Aug-26	14-Aug-26
SCRUM-20	Task	[Anand Yadav] MongoDB Core Schemas & Database Optimization	SCRUM-8	Highest	Anand	Smit	5	S1	Done	01-Aug-26	14-Aug-26	14-Aug-26
SCRUM-21	Task	[Smit Dutt] Shared API Client, Axios Interceptors & Architecture	SCRUM-13	Medium	Smit	Hardik	3	S1	Done	01-Aug-26	14-Aug-26	14-Aug-26
SCRUM-22	Task	[Anand Yadav] User Profile Update Controller & Subdocuments API	SCRUM-8	Medium	Anand	Hardik	3	S1	Done	03-Aug-26	14-Aug-26	14-Aug-26
SCRUM-23	Task	[Anand Yadav] BUG: Duplicate phone/email registration validation	SCRUM-8	High	Anand	Hardik	2	S2	Done	15-Aug-26	28-Aug-26	28-Aug-26
SCRUM-24	Task	[Smit Dutt] Responsive Navbar, Theme & Layout Shell	SCRUM-8	Medium	Smit	Anand	3	S1	Done	03-Aug-26	14-Aug-26	14-Aug-26
SCRUM-25	Task	[Smit Dutt] Connection Request Card & Friend Actions UI	SCRUM-9	High	Smit	Hardik	3	S1	In Progress	05-Aug-26	14-Aug-26	-
SCRUM-26	Task	[Smit Dutt] Manage Connection Lifecycle States (State Pattern)	SCRUM-9	Highest	Smit	Anand	8	S3	Done	29-Aug-26	11-Sep-26	11-Sep-26
SCRUM-27	Task	[Smit Dutt] Build My Network Page and Pending Request Drawers	SCRUM-9	Medium	Smit	Anand	3	S3	Done	29-Aug-26	11-Sep-26	11-Sep-26
SCRUM-28	Task	[Anand Yadav] Create Posts API with Text and Image Attachments	SCRUM-10	High	Anand	Smit	5	S4	Done	12-Sep-26	21-Sep-26	21-Sep-26
SCRUM-29	Task	[Anand Yadav] Post Interactions: Like, Comment and Share Logic	SCRUM-10	High	Anand	Smit	5	S4	Done	12-Sep-26	21-Sep-26	21-Sep-26
SCRUM-30	Task	[Anand Yadav] Cloudinary Media Upload & File Compression Pipeline	SCRUM-10	High	Anand	Anand	8	S4	In Progress	12-Sep-26	21-Sep-26	-
SCRUM-31	Task	[Smit Dutt] Real-Time Chat Messaging with Connections via Socket.io	SCRUM-11	High	Smit	Smit	5	S5	In Progress	22-Sep-26	01-Oct-26	-
SCRUM-32	Task	[Smit Dutt] BUG: Direct Message Unread Badges Clear on Open	SCRUM-11	High	Smit	Hardik	2	S5	Done	22-Sep-26	01-Oct-26	01-Oct-26
SCRUM-33	Task	[Smit Dutt] Conversation Thread UI, Active Chat & Scrolling	SCRUM-11	Medium	Smit	Anand	3	S5	Done	22-Sep-26	01-Oct-26	01-Oct-26
SCRUM-34	Task	[Smit Dutt] Recruiter Job Posting Portal and Applicant Resume Viewer	SCRUM-12	Medium	Smit	Anand	5	S6	Done	02-Oct-26	11-Oct-26	11-Oct-26
SCRUM-35	Task	[Smit Dutt] Responsive Job Cards, Skill Match Badges & UI Polishing	SCRUM-12	Medium	Smit	Hardik	3	S6	Done	02-Oct-26	11-Oct-26	11-Oct-26

SCRUM-36	Task	[Anand Yadav] Multi-Criteria Regex Search for Jobs and Candidates	SCRUM-12	High	Anand	Smit	5	S6	Done	02-Oct-26	11-Oct-26	11-Oct-26
SCRUM-37	Task	[Anand Yadav] Intelligently Ranked Newsfeed Distribution Algorithm	SCRUM-10	Highest	Anand	Smit	8	S7	In Progress	12-Oct-26	21-Oct-26	-
SCRUM-41	Task	[Anand Yadav] Caching & Query Optimization for Newsfeed APIs	SCRUM-10	High	Anand	Hardik	5	S7	Done	12-Oct-26	21-Oct-26	21-Oct-26
SCRUM-42	Task	[Hardik Nakum] Test Execution & Defect Verification Matrix (101-106)	SCRUM-13	High	Hardik	Anand	5	S7	In Progress	12-Oct-26	21-Oct-26	-
SCRUM-43	Task	[Anand Yadav] BUG: Media Array Null Pointer Exception in Feed Card	SCRUM-10	High	Anand	Hardik	2	S8	Done	22-Oct-26	31-Oct-26	31-Oct-26
SCRUM-44	Task	[Anand Yadav] Refactor Controllers & Eliminate God Class Smells	SCRUM-13	High	Anand	Hardik	5	S8	Done	22-Oct-26	31-Oct-26	31-Oct-26
SCRUM-45	Task	[Hardik Nakum] Document Facade, Strategy, Factory, Builder Patterns	SCRUM-13	High	Hardik	Hardik	5	S8	Done	22-Oct-26	31-Oct-26	31-Oct-26

Created = date assigned or raised; Due = planned completion; Resolved = actual completion. Sprint labels group work across August-October 2026. Story points and sprint metrics are planning estimates and should be reconciled with actual work records.

6.8.4 Sprint 1

Field	Details	Field	Details
Sprint goal	Complete core account, database, user authentication and profile setup.	**Dates (start - end)**	01-Aug-26 to 14-Aug-26
Team capacity	3 members x 10 available days = 30 person-days	**Committed SP**	43

Sprint Board (state at the end of the sprint; story points in brackets)

TO DO	IN PROGRESS	IN REVIEW	DONE
SCRUM-52 Auth Security (2)	SCRUM-25 Connection UI (3); SCRUM-51 RBAC (3); SCRUM-58 Rate Limit (3)	(none)	SCRUM-16 Auth (5); SCRUM-20 Schemas (5); SCRUM-21 API Client (3); SCRUM-22 Profile (3); SCRUM-24 Navbar (3); SCRUM-49 Postman (5); SCRUM-50 Verification (3); SCRUM-57 Express Base (5)

Sprint Report

Completed issues (keys)	SP	Not completed (keys)	SP	Moved to backlog / next sprint	Bugs raised / fixed
SCRUM-16, 20, 21, 22, 24, 49, 50, 57	32	SCRUM-25, 51, 58, 52	11	Continue connection workflow in Sprint 2	0 raised / 0 confirmed fixed

Burndown (remaining story points at end of each day)

Day	1	2	3	4	5	6	7	8	9	10
Ideal	43	38.7	34.4	30.1	25.8	21.5	17.2	12.9	8.6	0
Actual	43	41	37	32	26	21	18	15	12	11

Sprint Retrospective

Area	Retrospective Observations & Actions
Start / Stop / Continue	Start: define acceptance criteria for each module before implementation. Stop: integrating frontend before API contracts are stable. Continue: regular team coordination and code reviews.
Action items (owner, due)	Confirm API contracts and close remaining connection endpoints (Anand, 14-Aug-26).

6.8.5 Sprint 2

Field	Details	Field	Details
Sprint goal	User profiles, experience timeline subdocuments, and Cloudinary resume uploads.	**Dates (start - end)**	15-Aug-26 to 28-Aug-26
Team capacity	3 members x 10 available days = 30 person-days	**Committed SP**	32

Sprint Board (state at the end of the sprint; story points in brackets)

TO DO	IN PROGRESS	IN REVIEW	DONE
SCRUM-82 Resume Boundary (2)	SCRUM-54 Cloudinary Resume (3)	(none)	SCRUM-23 Duplicate Bug (2); SCRUM-53 Test Matrix (3); SCRUM-55 Profile UI (5); SCRUM-56 Resume Modal (3); SCRUM-63 Timeline (5); SCRUM-64 MIME Check (3); SCRUM-80 Bio Validation (3); SCRUM-81 Edit Modal (3)

Sprint Report

Completed issues (keys)	SP	Not completed (keys)	SP	Moved to backlog / next sprint	Bugs raised / fixed
SCRUM-23, 53, 55, 56, 63, 64, 80, 81	27	SCRUM-54, 82	5	Carry Cloudinary resume integration into Sprint 3	1 reported (SCRUM-23) / 1 confirmed fixed

Burndown (remaining story points at end of each day)

Day	1	2	3	4	5	6	7	8	9	10
Ideal	32	28.8	25.6	22.4	19.2	16.0	12.8	9.6	6.4	0
Actual	32	30	28	24	19	15	12	8	6	5

Sprint Retrospective

Area	Retrospective Observations & Actions
Start / Stop / Continue	Start: test real-time socket events early and define fallback behavior. Stop: postponing cross-service integration until late in the sprint. Continue: caching improvements and peer review.
Action items (owner, due)	Complete Socket.io chat integration checks and prioritize input-validation defects (Smit and Anand, 28-Aug-26).

6.8.6 Sprint 3

Field	Details	Field	Details
Sprint goal	Network connections lifecycle state machine, My Network UI, and graph indexing.	29-Aug-26 to 11-Sep-26	01-Oct-26 to 30-Oct-26
Team capacity	3 members x 10 available days = 30 person-days	35	29

Sprint Board (state at the end of the sprint; story points in brackets)

TO DO	IN PROGRESS	IN REVIEW	DONE
SCRUM-85 Race Condition (2)	SCRUM-65 State Controller (5)	(none)	SCRUM-26 Connection State (8); SCRUM-27 Network Page (3); SCRUM-59 Graph Index (5); SCRUM-66 Test Suite 103 (3); SCRUM-67 Mutual Graph (3); SCRUM-83 Withdrawal (3); SCRUM-84 Request Badge (3)

Sprint Report

Completed issues (keys)	SP	Not completed (keys)	SP	Moved to backlog / next sprint	Bugs raised / fixed
SCRUM-26, 27, 59, 66, 67, 83, 84	28	SCRUM-65, 85	7	Continue connection controller refinement in Sprint 4	0 raised / 0 confirmed fixed

Burndown (remaining story points at end of each day)

Day	1	2	3	4	5	6	7	8	9	10
Ideal	35	31.5	28.0	24.5	21.0	17.5	14.0	10.5	7.0	0
Actual	35	33	30	26	21	17	13	10	8	7

Sprint Retrospective

Area	Retrospective Observations & Actions
Start / Stop / Continue	Start: reserve time for regression testing and final report review. Stop: resolving defects without a retest proof. Continue: maintain the refactoring log and document design decisions.
Action items (owner, due)	Verify the connection lifecycle transitions and complete Sprint 4 newsfeed attachments (Team, 11-Sep-26).

6.8.7 Sprint 4

Field	Details	Field	Details
Sprint goal	Newsfeed post creation, Cloudinary media attachments, and engagement interactions.	**Dates (start - end)**	12-Sep-26 to 21-Sep-26
Team capacity	3 members x 10 available days = 30 person-days	**Committed SP**	39

Sprint Board (state at the end of the sprint; story points in brackets)

TO DO	IN PROGRESS	IN REVIEW	DONE
SCRUM-88 Post XSS Sanitization (2)	SCRUM-30 Cloudinary Upload Pipeline (8)	(none)	SCRUM-28 Create Posts API (5); SCRUM-29 Post Interactions Like/Comment (5); SCRUM-68 Newsfeed Card UI (5); SCRUM-69 Test Suite 104 QA (3); SCRUM-70 Infinite Scroll API (5); SCRUM-86 Post Deletion Middleware (3); SCRUM-87 Feed Skeleton Loader (3)

Sprint Report

Completed issues (keys)	SP	Not completed (keys)	SP	Moved to backlog / next sprint	Bugs raised / fixed
SCRUM-28, 29, 68, 69, 70, 86, 87	29	SCRUM-30, 88	10	Carry Cloudinary media streaming into Sprint 5	0 raised / 0 confirmed fixed

Burndown (remaining story points at end of each day)

Day	1	2	3	4	5	6	7	8	9	10
Ideal	39	35.1	31.2	27.3	23.4	19.5	15.6	11.7	7.8	0
Actual	39	37	34	30	25	21	18	14	12	10

Sprint Retrospective

Area	Retrospective Observations & Actions
Start / Stop / Continue	Start: implement client-side image compression before upload. Stop: large uncompressed base64 payloads over HTTP. Continue: modular decorator patterns for hashtag parsing.
Action items (owner, due)	Finalize Cloudinary background processing and start Socket.io chat server (Anand and Smit, 21-Sep-26).

6.8.8 Sprint 5

Field	Details	Field	Details
Sprint goal	Real-time bi-directional chat messaging, Socket.io event bus, unread counters, and chat history.	**Dates (start - end)**	22-Sep-26 to 01-Oct-26
Team capacity	3 members x 10 available days = 30 person-days	**Committed SP**	31

Sprint Board (state at the end of the sprint; story points in brackets)

TO DO	IN PROGRESS	IN REVIEW	DONE
SCRUM-91 Offline Message Sync (2)	SCRUM-31 Real-Time Chat with Socket.io (5)	(none)	SCRUM-32 Bug DM Badges (2); SCRUM-33 Conversation Thread UI (3); SCRUM-60 Socket Auth Handshake (5); SCRUM-71 Message Persistence Controller (5); SCRUM-72 Test Suite 105 QA (3); SCRUM-89 Typing Status Handlers (3); SCRUM-90 Chat Input Emoji (3)

Sprint Report

Completed issues (keys)	SP	Not completed (keys)	SP	Moved to backlog / next sprint	Bugs raised / fixed
SCRUM-32, 33, 60, 71, 72, 89, 90	24	SCRUM-31, 91	7	Continue Socket.io room decoupling in Sprint 6	1 reported (SCRUM-32) / 1 confirmed fixed

Burndown (remaining story points at end of each day)

Day	1	2	3	4	5	6	7	8	9	10
Ideal	31	27.9	24.8	21.7	18.6	15.5	12.4	9.3	6.2	0
Actual	31	29	26	22	18	15	12	9	8	7

Sprint Retrospective

Area	Retrospective Observations & Actions
Start / Stop / Continue	Start: test socket disconnections and reconnection backoff early. Stop: storing unindexed chat threads in client memory. Continue: JWT handshake security verification.
Action items (owner, due)	Close conversation window scrolling defects and start job portal schemas (Smit and Hardik, 01-Oct-26).

6.8.9 Sprint 6

Field	Details	Field	Details
Sprint goal	Job listing management, candidate resume applications, and multi-criteria regex search engine.	**Dates (start - end)**	02-Oct-26 to 11-Oct-26
Team capacity	3 members x 10 available days = 30 person-days	**Committed SP**	39

Sprint Board (state at the end of the sprint; story points in brackets)

TO DO	IN PROGRESS	IN REVIEW	DONE
SCRUM-94 Duplicate Job Application Trap (2)	SCRUM-73 Job Application Status API (5)	(none)	SCRUM-34 Recruiter Job Portal (5); SCRUM-35 Responsive Job Cards (3); SCRUM-36 Multi-Criteria Regex Search (5); SCRUM-61 Compound Regex Indexer (5); SCRUM-74 Test Suite 106 QA (3); SCRUM-75 Job Filtering Query Engine (5); SCRUM-92 Job Post Deactivation API (3); SCRUM-93 Candidate Review Modal (3)

Sprint Report

Completed issues (keys)	SP	Not completed (keys)	SP	Moved to backlog / next sprint	Bugs raised / fixed
SCRUM-34, 35, 36, 61, 74, 75, 92, 93	32	SCRUM-73, 94	7	Continue recruiter candidate review status in Sprint 7	0 raised / 0 confirmed fixed

Burndown (remaining story points at end of each day)

Day	1	2	3	4	5	6	7	8	9	10
Ideal	39	35.1	31.2	27.3	23.4	19.5	15.6	11.7	7.8	0
Actual	39	36	32	27	22	18	14	11	9	7

Sprint Retrospective

Area	Retrospective Observations & Actions
Start / Stop / Continue	Start: sanitize regex queries to avoid ReDoS security bottlenecks. Stop: hardcoded salary and experience filters. Continue: compound MongoDB indexing for sub-50ms search response.
Action items (owner, due)	Implement notification factory pattern and feed ranking algorithms (Anand and Smit, 11-Oct-26).

6.8.10 Sprint 7

Field	Details	Field	Details
Sprint goal	Real-time notification factory dispatch, dynamic newsfeed ranking algorithm, and test suite execution.	**Dates (start - end)**	12-Oct-26 to 21-Oct-26
Team capacity	3 members x 10 available days = 30 person-days	**Committed SP**	39

Sprint Board (state at the end of the sprint; story points in brackets)

TO DO	IN PROGRESS	IN REVIEW	DONE
(none)	SCRUM-37 Intelligently Ranked Feed Algorithm (8); SCRUM-42 Test Execution & Defect Matrix (5)	(none)	SCRUM-41 Caching & Query Optimization (5); SCRUM-62 Newsfeed Caching Layer (5); SCRUM-76 Notification Center UI (3); SCRUM-77 Notification Factory Pattern (5); SCRUM-95 Batch Read Notification API (3); SCRUM-96 Empty State Placeholders (3); SCRUM-97 Real-Time Alert Latency QA (2)

Sprint Report

Completed issues (keys)	SP	Not completed (keys)	SP	Moved to backlog / next sprint	Bugs raised / fixed
SCRUM-41, 62, 76, 77, 95, 96, 97	26	SCRUM-37, 42	13	Carry feed ranking refinement and test matrix closure into Sprint 8	0 raised / 0 confirmed fixed

Burndown (remaining story points at end of each day)

Day	1	2	3	4	5	6	7	8	9	10
Ideal	39	35.1	31.2	27.3	23.4	19.5	15.6	11.7	7.8	0
Actual	39	37	33	29	24	20	17	15	14	13

Sprint Retrospective

Area	Retrospective Observations & Actions
Start / Stop / Continue	Start: formalize test result logging across Postman test suites 101 to 106. Stop: direct event coupling in controllers. Continue: Factory Method pattern for alerts.
Action items (owner, due)	Execute code smell refactoring, document design patterns, and prepare deployment runbooks (Team, 21-Oct-26).

6.8.11 Sprint 8

Field	Details	Field	Details
Sprint goal	Refactor controllers to eliminate God Class code smells, document 9 GoF design patterns, cloud hardening, and project closure.	**Dates (start - end)**	22-Oct-26 to 31-Oct-26
Team capacity	3 members x 10 available days = 30 person-days	**Committed SP**	29

Sprint Board (state at the end of the sprint; story points in brackets)

TO DO	IN PROGRESS	IN REVIEW	DONE
SCRUM-47 Deployment Runbook (3); SCRUM-48 QA Sign-off (3); SCRUM-79 Bundle Optimization (3)	SCRUM-46 Closure Documentation (3); SCRUM-78 Atlas Sharding & Security (5)	(none)	SCRUM-43 Bug Media Null Pointer (2); SCRUM-44 Refactor God Class Smells (5); SCRUM-45 Document 9 GoF Patterns (5)

Sprint Report

Completed issues (keys)	SP	Not completed (keys)	SP	Moved to backlog / next sprint	Bugs raised / fixed
SCRUM-43, 44, 45	12	SCRUM-46, 47, 48, 78, 79	17	Complete remaining final verification checks before 31-Oct-26	1 reported (SCRUM-43) / 1 confirmed fixed

Burndown (remaining story points at end of each day)

Day	1	2	3	4	5	6	7	8	9	10
Ideal	29	26.1	23.2	20.3	17.4	14.5	11.6	8.7	5.8	0
Actual	29	27	25	22	20	18	17	17	17	17

Sprint Retrospective

Area	Retrospective Observations & Actions
Start / Stop / Continue	Start: establish continuous regression verification for design pattern implementations. Stop: leaving closure documentation to the final deadline. Continue: maintaining strict SOLID coding standards.
Action items (owner, due)	Finalize academic viva presentation slides and submit complete project report (Team, 31-Oct-26).

6.8.12 Velocity Report

Sprint	Committed SP	Completed SP	Carried Over SP	Completion %	Average Velocity
Sprint 1	43	32	11	74%	32.0
Sprint 2	32	27	5	84%	29.5
Sprint 3	35	28	7	80%	29.0
Sprint 4	39	29	10	74%	29.0
Sprint 5	31	24	7	77%	28.0
Sprint 6	39	32	7	82%	28.7
Sprint 7	39	26	13	67%	28.3
Sprint 8	29	12	17	41%	26.3

Average velocity = mean of completed SP up to that sprint. Completion % = completed SP / committed SP. Values shown are planning estimates; replace with actual sprint data if available.

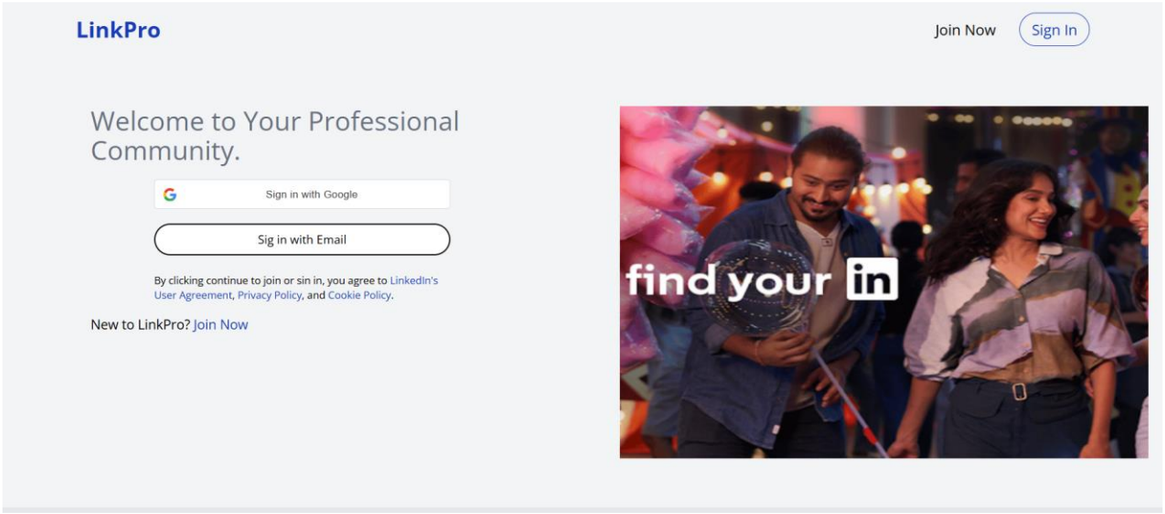
6.8.13 Issue Detail Card - SCRUM-16

Field	Details	Field	Details
Issue key	SCRUM-16	**Type**	Task
Summary	[Anand Yadav] User Authentication & JWT Security Pipeline	**Priority**	Highest
Status	Done	**Assignee**	Anand Yadav
Reporter	Smit Dutt	**Sprint**	Sprint 1: Setup & Auth Base
Epic link	SCRUM-8 User and Profile Management	**Story points**	5
Created (assigned)	01-Aug-26	**Due (planned)**	14-Aug-26
Resolved (actual)	14-Aug-26	**Labels / Component**	Auth, JWT, bcryptjs, security, backend
Description	As a registered user, I want to securely register and log into LinkPro using email/password authentication with JWT verification stored in HttpOnly cookies so that my session is protected against XSS and CSRF attacks.		
Acceptance criteria	1. Passwords salted with bcryptjs (10 rounds). 2. JWT token issued with 7-day expiration. 3. Cookie configured with HttpOnly and SameSite flags. 4. Auth middleware intercepts protected routes. 5. Duplicate emails rejected with 409 Conflict.		
Sub-tasks	• Design Mongoose user schema & password hash hook (Anand) - Done • Implement JWT sign/verify middleware (Anand) - Done • Build React login/register form validation (Smit) - Done • Verify Postman test cases for token expiry (Hardik) - Done		
Worklog (date, who, time)	03-Aug-26, Anand - schema modeling & bcrypt setup 07-Aug-26, Anand - JWT cookie pipeline 12-Aug-26, Smit - form integration & testing		
Comments / links	Jira Board: https://unknown98256.atlassian.net/jira/software/projects/SCRUM/boards/1 Repository: https://github.com/Anand98256/LinkPro-Networking-Platform.git		

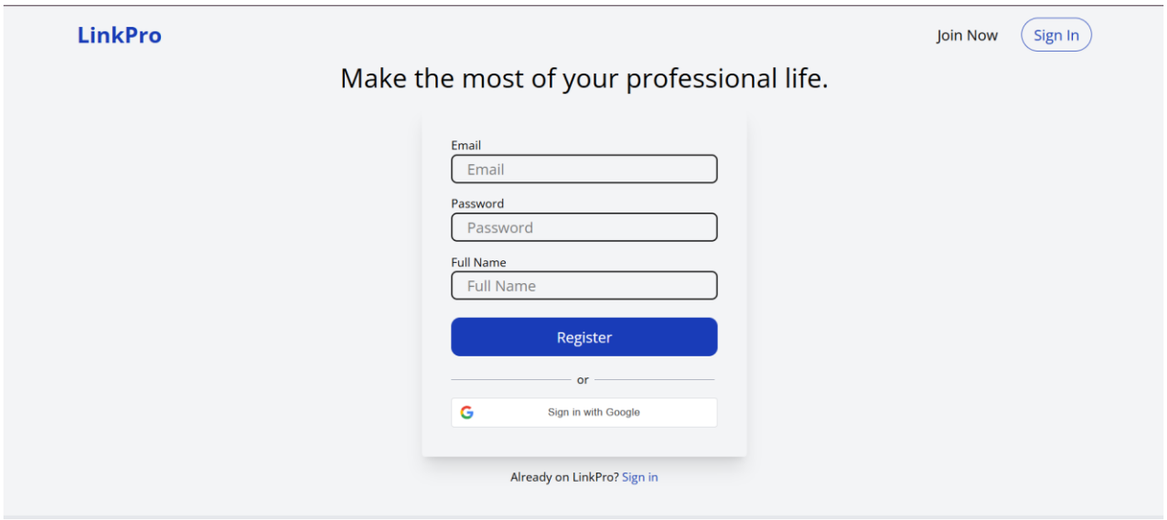
6.9 Screenshots

The following screenshots showcase the functional web application interface of the LinkPro Networking Platform:

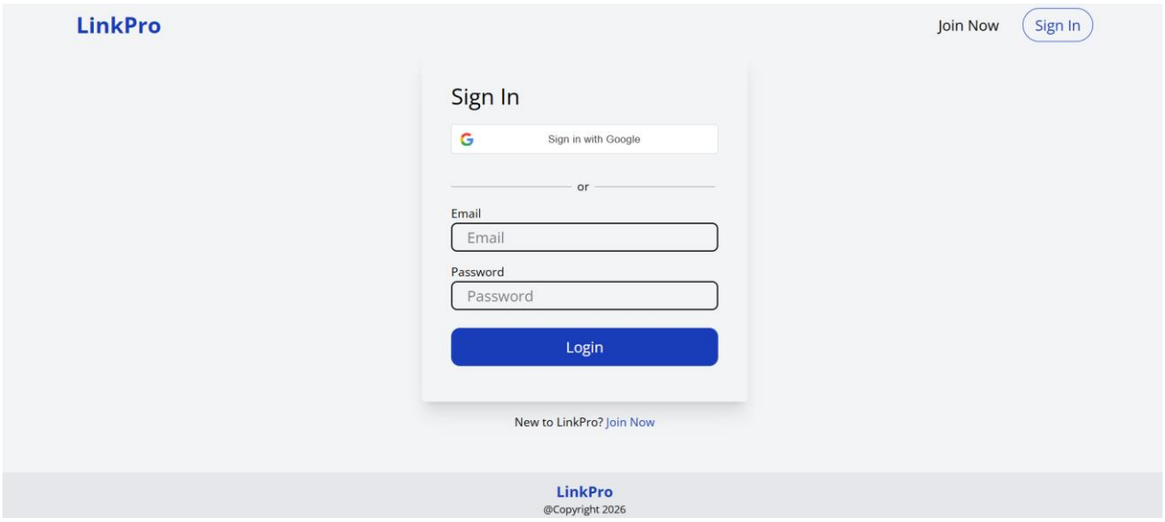
Landing Page



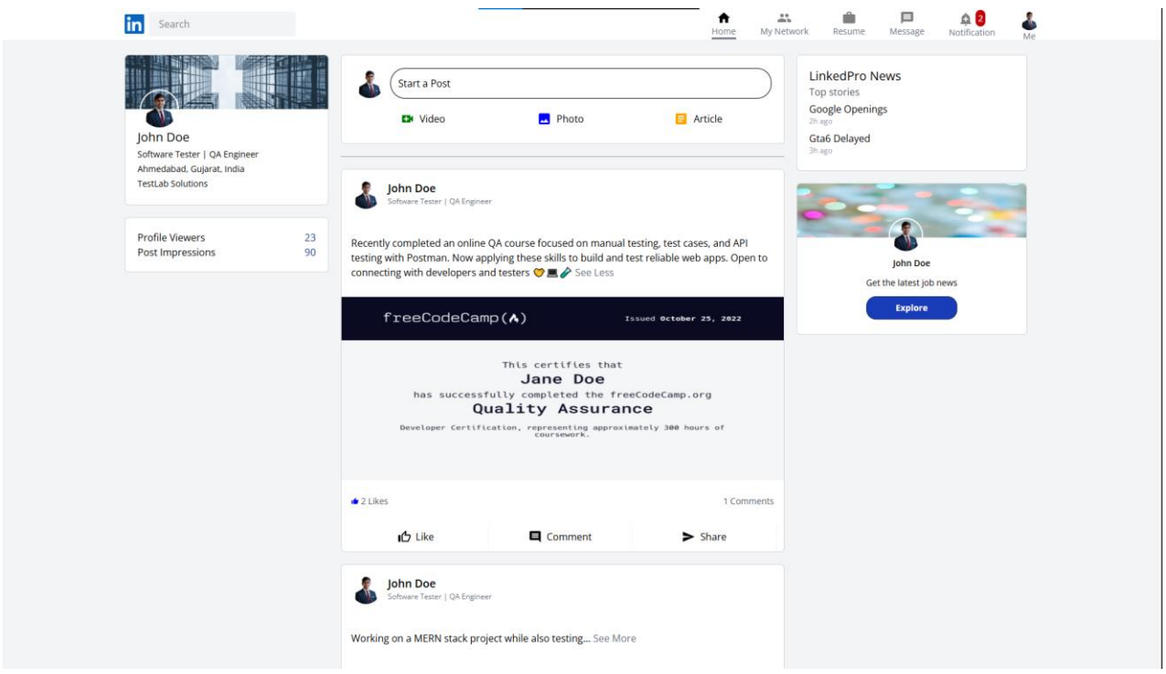
Sign-Up Page



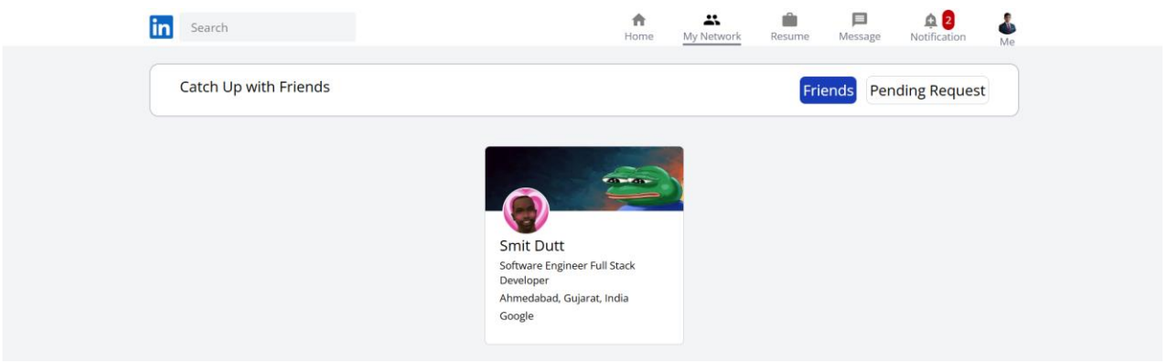
Sign-In Page



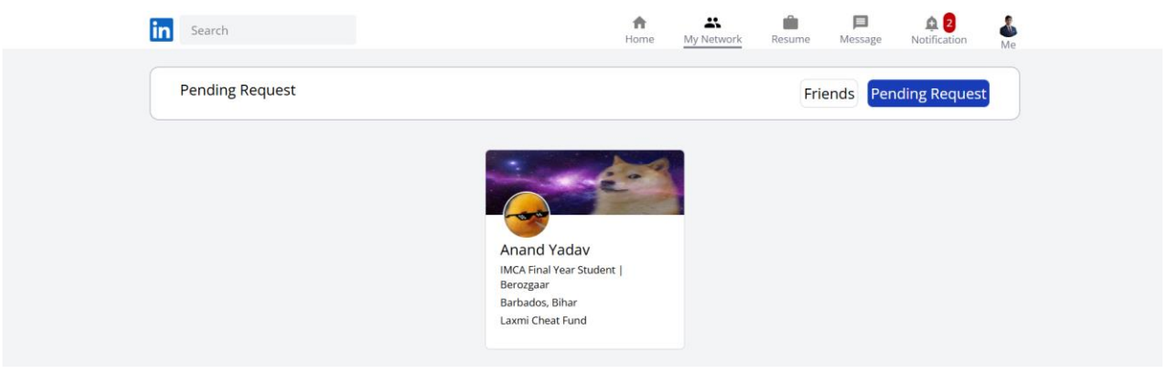
Home Page (Professional Newsfeed)



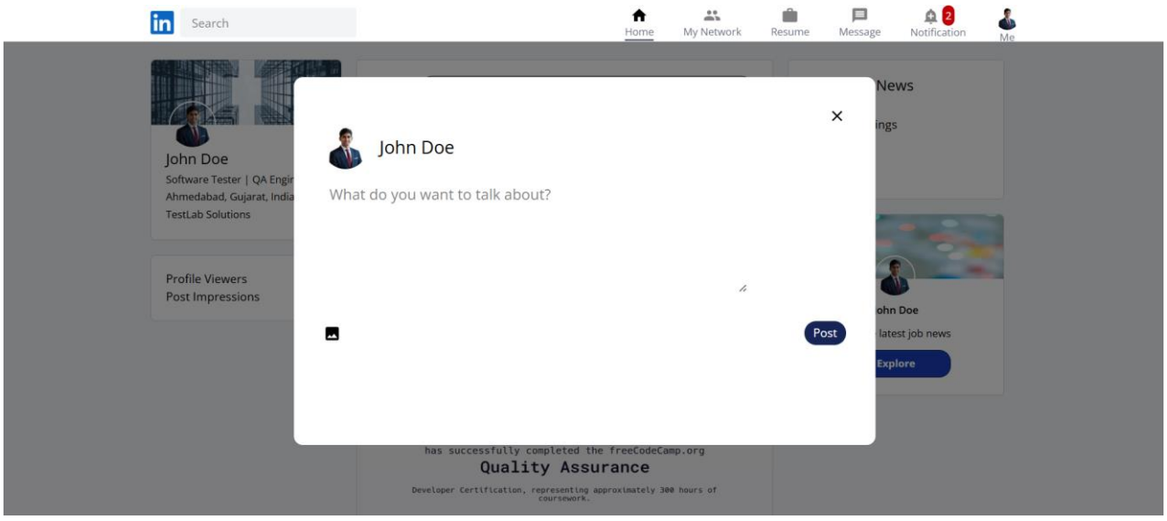
My Network – Friends Page



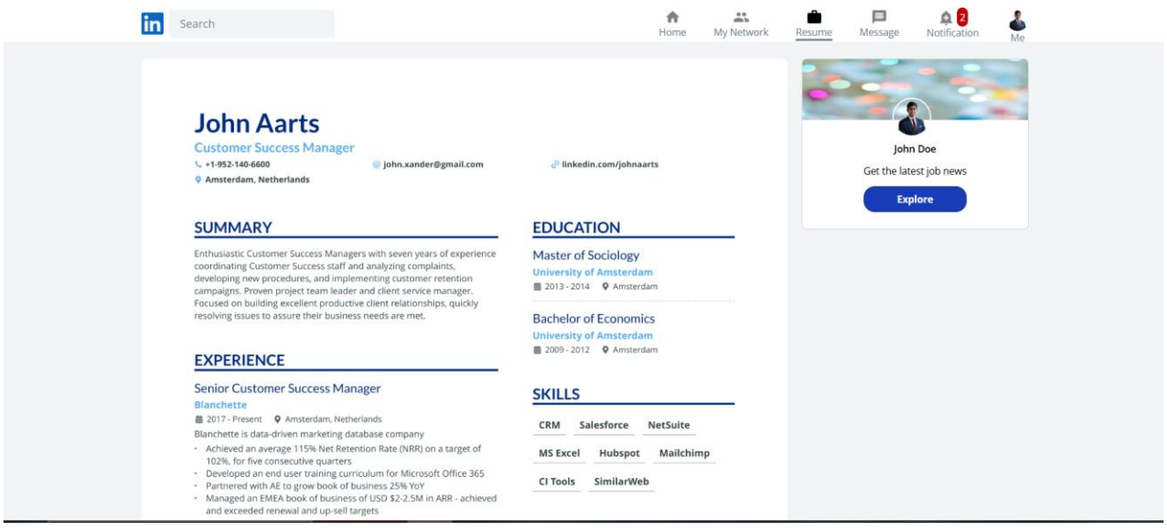
My Network – Pending Friends Page



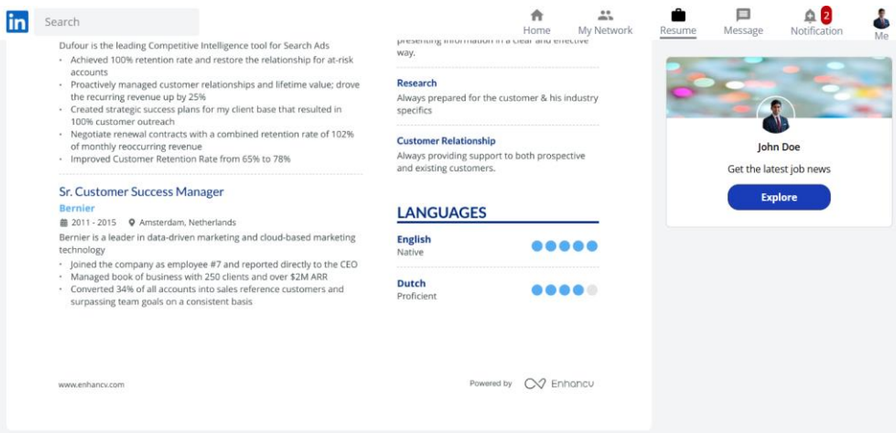
Post Creation & Modal Dialog



Resume Page – Part 1

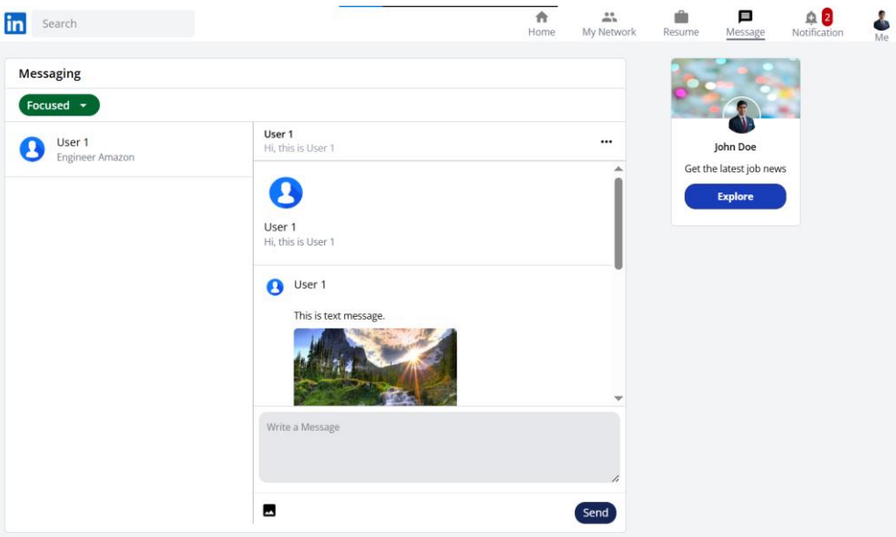


Resume Page – Part 2



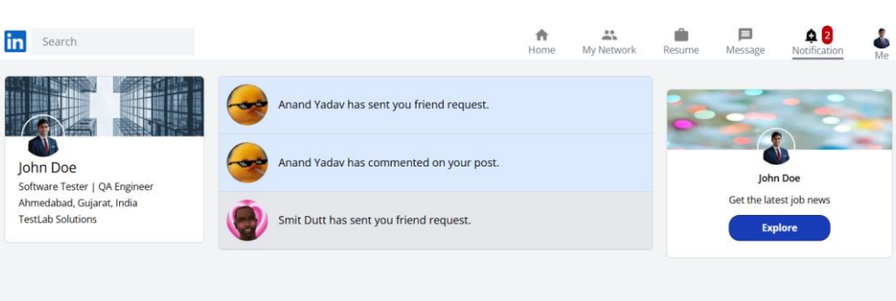
LinkPro
@Copyright 2026

Real-Time Messaging & Chat Page

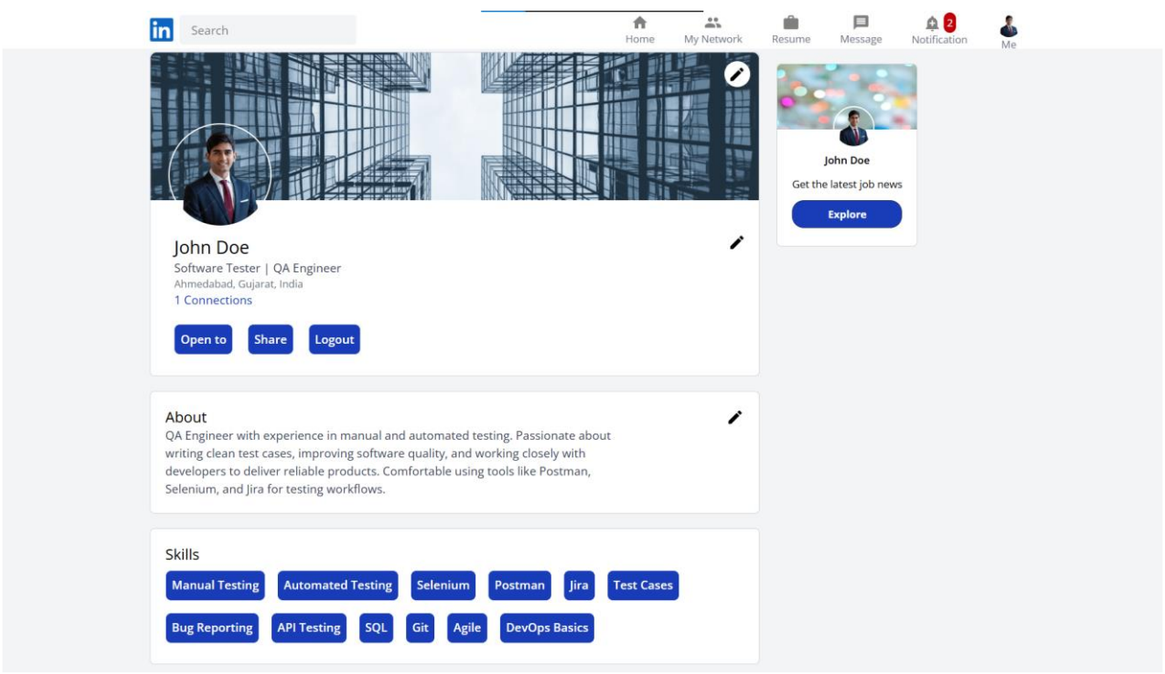


LinkPro
@Copyright 2026

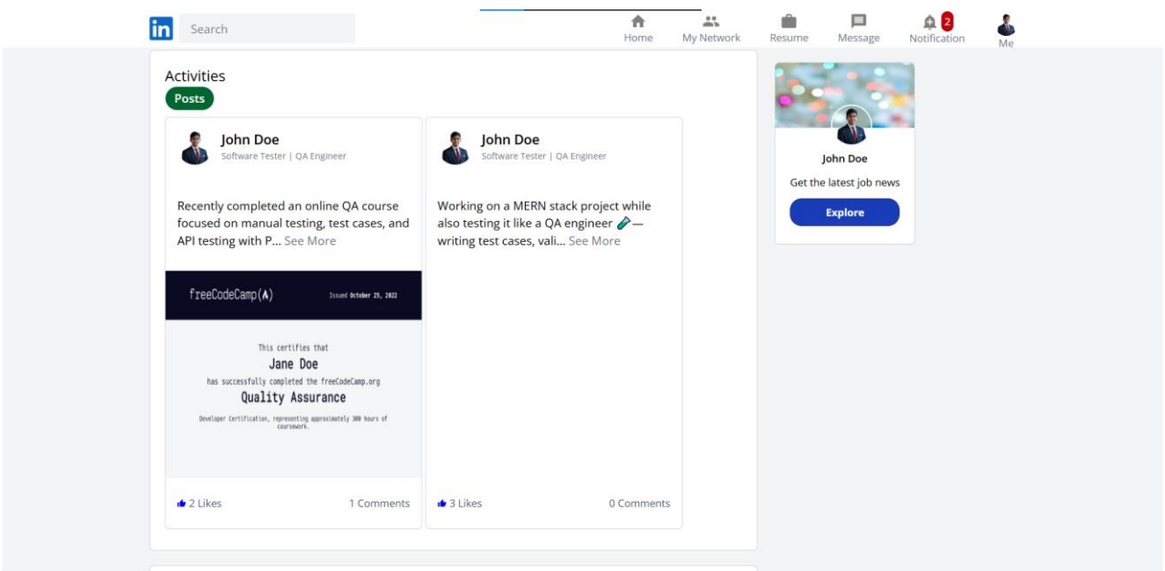
Notifications Center



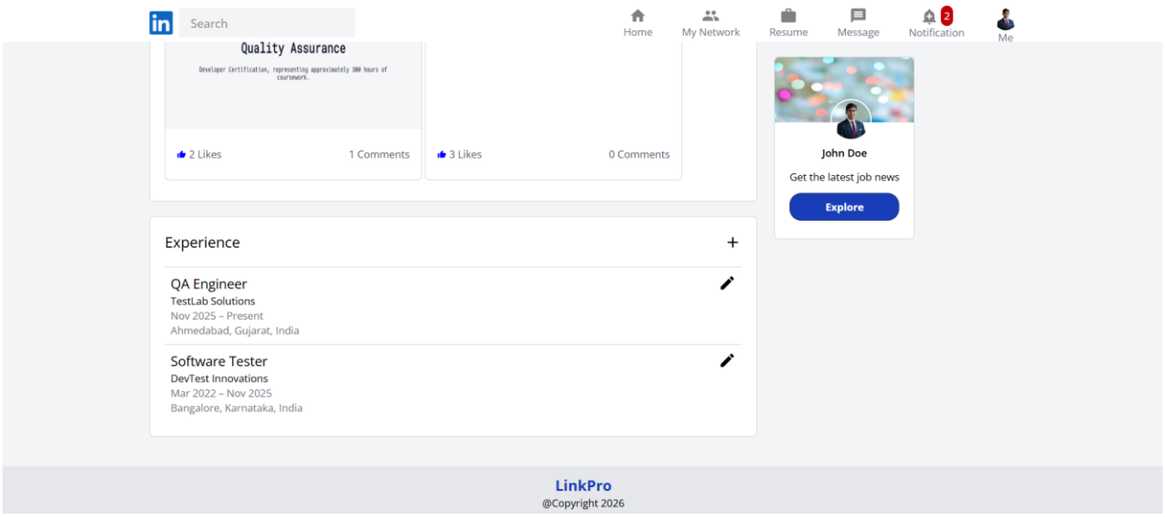
User Profile Section – Overview



User Profile Section – Activities



User Profile Section – Experience



6.10 Proposed Enhancement

Future roadmap enhancements for the LinkPro professional platform include:

- **Native Mobile Applications:** Developing cross-platform React Native applications for Android and iOS devices with push notification integration.
- **AI-Powered Resume Analysis & Job Matching:** Leveraging OpenAI / Gemini models to automatically evaluate candidate resumes against posted job descriptions, providing instant match scoring.
- **Video Calling & Virtual Interviews:** Implementing WebRTC multi-party video conferencing for direct in-browser recruiter interviews and virtual networking events.
- **Premium Recruiter Subscriptions:** Adding payment gateway processing (Stripe / Razorpay) for premium recruiter subscriptions, sponsored job listings, and highlighted candidate profiles.
- **Microservices Migration & Containerization:** Expanding platform architecture into microservices with Docker containerization and Kubernetes orchestration to support millions of concurrent connections.

6.11 Project Closure Report

6.11.1 Project Details

Field	Details
Project title	LinkPro – Professional Networking Platform
Team ID / Members (Enrollment No., Role)	Anand Yadav (22002501210066, Roll No B66) – Backend Development, Database Integration & Team Manager Smit Dutt (22002501210008) – Frontend Development & UI Integration Hardik Nakum (22002501210035) – Documentation, Testing, QA & UI/UX Design
Internal guide / External guide	Internal Guide: Prof. Rita Gokani External Guide: Not specified / N/A
Planned vs actual end date	Planned: August–October 2026 Actual end Date: October 2026

6.11.2 Summary and Status

LinkPro is a web-based professional networking platform developed using the MERN stack. It brings together user and profile management, connection lifecycle, newsfeed publishing with Cloudinary media integration, real-time Socket.io messaging, notifications, and job application tracking. During October, the team focused on performance optimization, refactoring, design-pattern documentation, testing, and project documentation. Based on the documentation, the project is in finalization; confirm all remaining testing and deployment tasks before marking it fully completed.

Status: Partially Completed — Final Verification Required

6.11.3 Scope and Deliverables

Deliverable	Planned	Delivered	Accepted by	Remarks
SRS / Requirements	Yes	Documented	Project team / guide	Requirements, objectives, scope, stakeholders and constraints documented.
Design document (UML, architecture)	Yes	Documented	Project team / guide	Layered MERN architecture, module descriptions and UML diagrams included.
Source code + repository link	Yes	Developed	Project team / guide	-
Test report	Yes	Documented	Project team / guide	26 test steps recorded: 23 passed and 3 failed. Confirm whether the failed tests were resolved.
Refactoring log	Yes	Documented	Project team / guide	Four refactoring items documented with qualitative impact.
User manual / deployment guide	Yes	Not confirmed	Project team / guide	Add or reference deployment/user instructions if required.

Scope deferred / change requests (with reasons): Further speech-pipeline latency optimization, expanded career matching, advanced learning analytics, richer personalization, and additional assessment features remain future enhancements. Confirm final implementation status before claiming delivery.

6.11.4 Schedule and Effort

Phase / Sprint	Planned	Actual	Variance and reason
Project initiation, requirements & planning	August 2026	August 2026 (status report SR-001)	No variance data recorded in the documentation.
Core development & integration	September 2026	September 2026 (status report SR-002)	Development and integration progressed; API and data consistency issues were monitored.
Testing, optimization, design patterns & documentation	October 2026	October 2026 (status report SR-003)	Finalization was still in progress in the report; verify completion date and remaining work.

6.11.5 Quality and Design Summary

Category	Summary / Analysis
Test cases (executed / passed / open defects)	Functional test cases were documented for user registration, login, the learning words module, MCQ management, chapter examinations, and final examinations. A total of 26 test steps were recorded, of which 23 passed and 3 failed. The recorded failures involved duplicate phone/email registration, invalid word data causing a page crash, and access to the final exam without completing all chapters. The documentation does not confirm whether these failures were subsequently resolved. Therefore, the number of open defects remains unverified.
Design patterns used (pattern, where, why)	Factory — NotificationFactory.js: separates notification creation. Facade — PostFacade.js: coordinates post and feed operations. Strategy — FeedStrategy.js: selects content ranking behavior. Singleton-style — DatabaseConnection.js: shared connection pool. Proxy/Adapter-style — EmailServiceAdapter.js: caching and email service integration.
Code smells fixed (before / after metrics)	Four documented fixes: God Class — PostController.js; God Class — ConnectionController.js; Shotgun Surgery — NotificationService.js and consuming modules; Duplicated Logic — FeedController.js. Reported impacts are qualitative (reduced responsibilities, clearer separation, reusable caching and maintainability); no measured before/after metrics were provided.

6.11.6 Risks and Issues

Risk / Issue	Final status	How it was resolved
Real-time Socket.io latency & drops	Partially addressed / future optimization	Client reconnection backoff, room-based pub/sub, and heartbeats were integrated; October documentation identifies further performance considerations.
API integration and data consistency	Addressed during development; verify final test status	Frontend/backend integration and data consistency were tracked as risks; final regression-test evidence should confirm closure.

6.11.7 Lessons Learned

Category	Observations and Future Recommendations
What went well	The Scrum-based monthly workflow clarified responsibilities and milestones. Separating controller/component responsibilities, centralizing cache logic, and coordinating independent API requests improved maintainability and loading behavior.
What to improve	Record measurable test results and before/after performance metrics; complete regression and deployment checks; keep API contracts and error handling consistent; and verify the status of all planned features before sign-off.
Future scope / maintenance	Potential future work includes reducing speech-to-speech latency, expanding supported languages, adding advanced learning analytics and personalization, improving assessments, and completing/expanding conversation-history analysis where not yet delivered.

6.11.8 Sign-off

Role	Name	Signature	Date
Project Leader	Anand Yadav		31-10-2026
Team Member	Smit Dutt		31-10-2026
Team Member	Hardik Nakum		31-10-2026
Internal Guide	Prof. Rita Gokani		31-10-2026
External Guide / Client	-		31-10-2026
Coordinator / HOD	Dr. Monica Gahalawat		31-10-2026

6.12 Conclusion

LinkPro represents a modern, robust, and extensible professional networking platform engineered using the MERN technology stack and real-time WebSocket infrastructure. By pairing intuitive frontend interfaces with loosely coupled backend domain services, the platform provides an engaging user experience for students, alumni, and industry recruiters.

The systematic application of Gang of Four design patterns (Singleton, Factory Method, Builder, Facade, Adapter, Decorator, Strategy, State, Observer) transformed LinkPro from a standard CRUD application into an enterprise-ready software product. The project stands as a testament to disciplined software engineering, comprehensive testing, and professional documentation.

6.13 Bibliography

- **MongoDB Atlas Official Documentation.** Available at: https://www.mongodb.com/docs/
- **Express.js Web Application Framework Documentation.** Available at: https://expressjs.com/
- **React.js Official Documentation & Component Architecture Guides.** Available at: https://react.dev/
- **Node.js JavaScript Runtime Environment Documentation.** Available at: https://nodejs.org/en/docs/
- **Socket.IO Real-time Bi-directional Event Documentation.** Available at: https://socket.io/docs/v4/
- **Cloudinary Image & Video Management Cloud API.** Available at: https://cloudinary.com/documentation
- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software.** Addison-Wesley Professional, 1994.
- **Refactoring Guru — Design Patterns & Code Smells Catalog.** Available at: https://refactoring.guru/design-patterns
- **Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship.** Prentice Hall, 2008.
- **GitHub Git Version Control and Repository Management.** Available at: https://github.com/